Lecture 11:

Long Short-Term Memory Networks (LSTMs) – Part I

Markus Hohle

University California, Berkeley

Machine Learning Algorithms

MSSE 277B, 3 Units

Course Map

Lecture 1: Course Overview and Introduction to Machine Learning

Lecture 2: Bayesian Methods in Machine Learning

Lecture 3: Dimensionality Reduction: Principal Component Analysis

Lecture 4: Linear and Non-linear Regression and Classification

Lecture 5: Unsupervised Learning: K-Means, GMM, Trees

Lecture 6: Adaptive Learning and Gradient Descent Optimization Algorithms

classic ML tools & algorithms

Lecture 7: Introduction to Artificial Neural Networks - The Perceptron

Lecture 8: Introduction to Artificial Neural Networks - Building Multiple Dense Layers

Lecture 9: Convolutional Neural Networks (CNNs) - Part I

Lecture 10: CNNs - Part II

Lecture 11: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs)

Lecture 12: Combining LSTMs and CNNs

Lecture 13: Running Models on GPUs and Parallel Processing

Lecture 14: Project Presentations

Lecture 15: Transformer

Lecture 16: GNN

ANNs/AI/Deep Learning

https://www.analyticsvidhya.com

Outline

- Idea and classic RNNs
- LSTMs
- *B*ack*P*ropagation *T*hrough *T*ime (BPTT)
- Syntax and some examples

https://www.analyticsvidhya.com

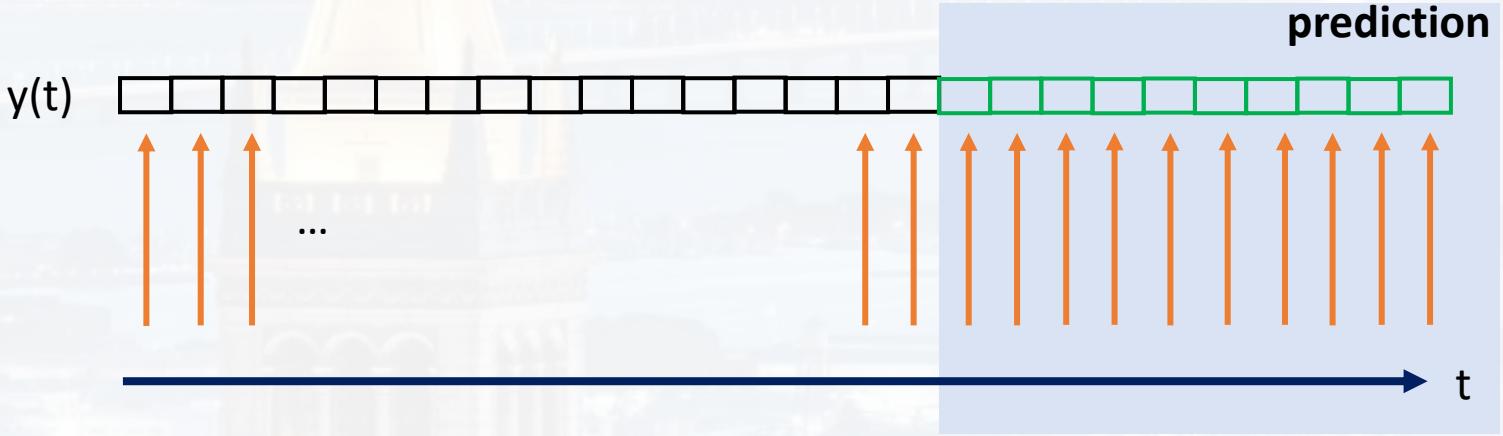
Outline

- **Idea and classic RNNs**
- LSTMs
- *B*ack*P*ropagation *T*hrough *T*ime (BPTT)
- Syntax and some examples

LSTMs:

- **R**ecurrent **N**eural **N**etwork

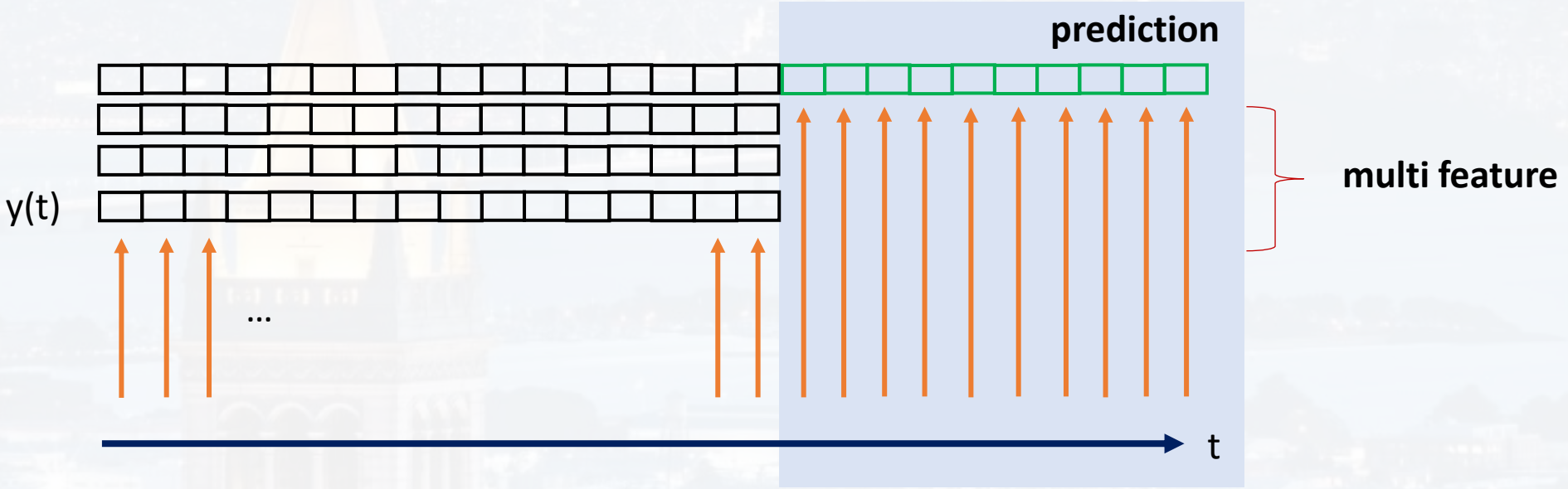
- time series analysis regression (**prediction** and **forecasting**)
- first step towards GenAI
- time series analysis classification
- early speech recognition
- handwriting
- “precursor” of LSTMs
- invented by **Shun'ichi Amari** in 1972



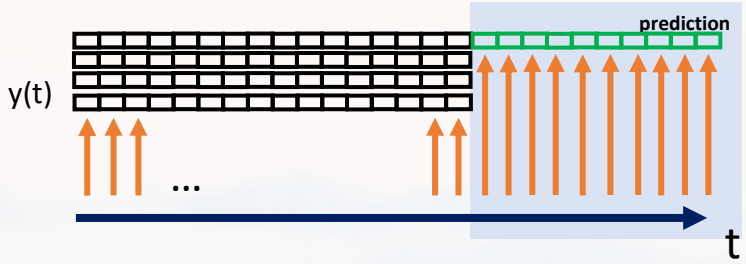
LSTMs:

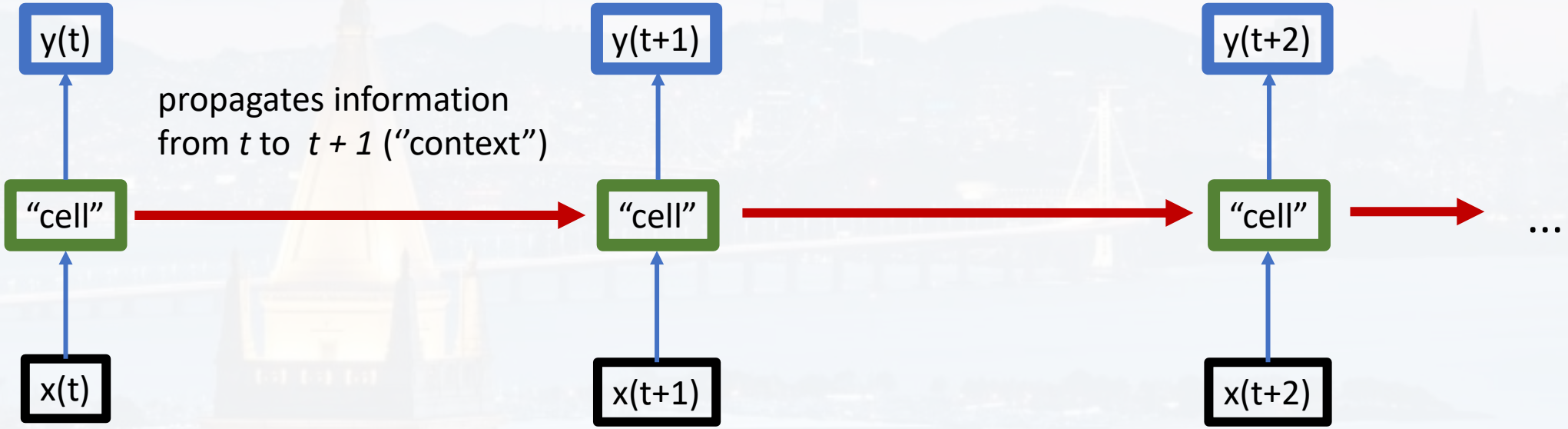
- **R**ecurrent **N**eural **N**etwork

- time series analysis regression (**prediction** and **forecasting**)
- first step towards GenAI
- time series analysis classification
- early speech recognition
- handwriting
- “precursor” of LSTMs
- invented by **Shun'ichi Amari** in 1972



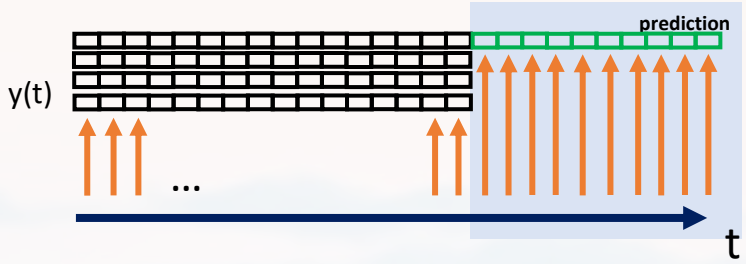
LSTMs:

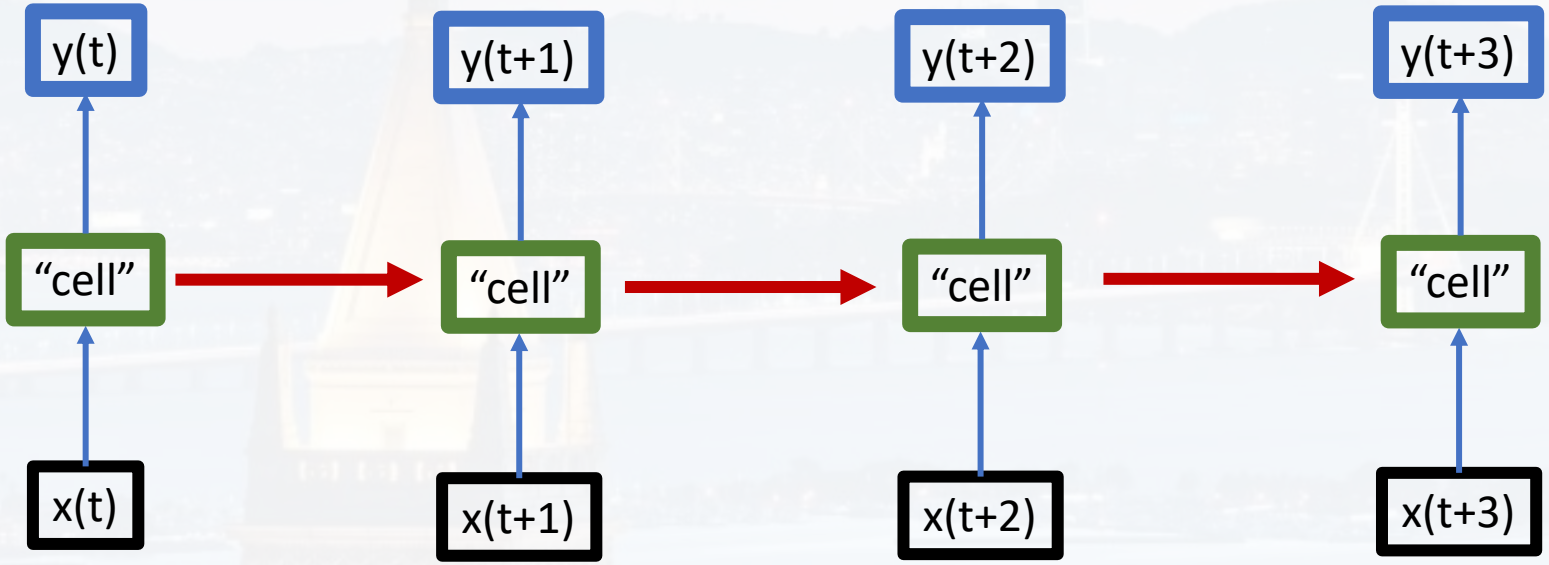

prediction
y(t)
...
t


y(t)
y(t+1)
y(t+2)
propagates information
from *t* to *t + 1* ("context")
"cell"
"cell"
"cell"
...
x(t)
x(t+1)
x(t+2)

LSTMs:

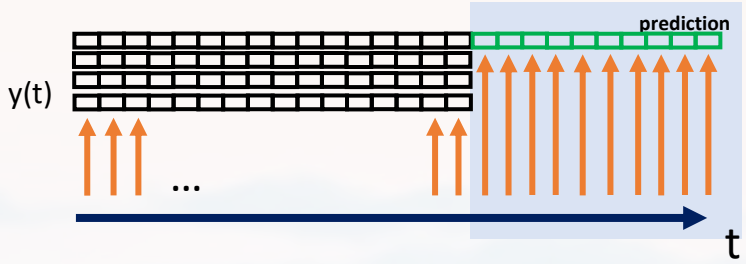
"many to many"

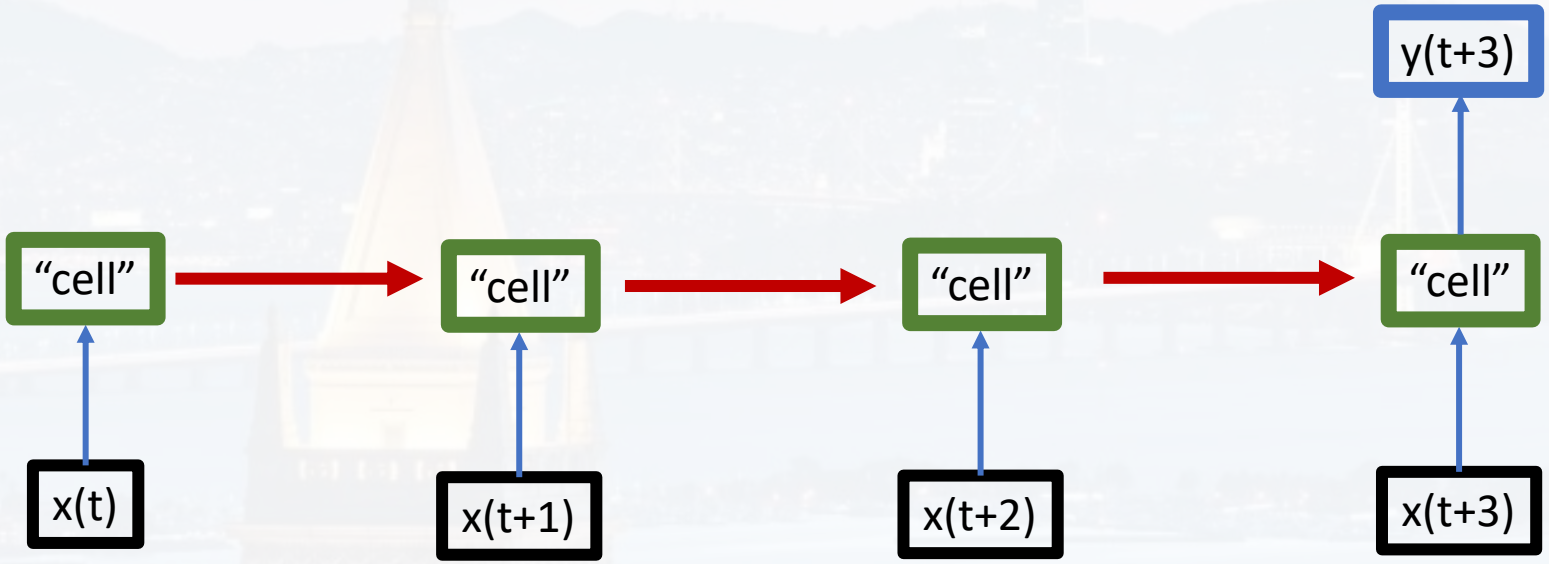




LSTMs:

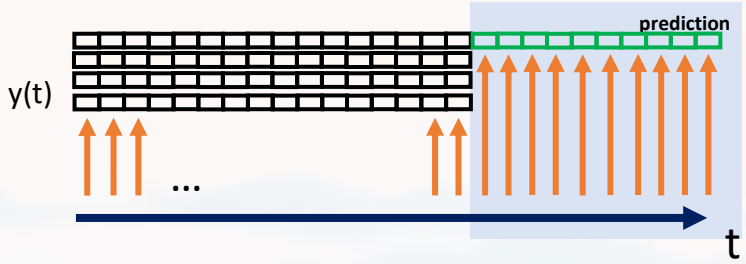
"many to one"
(classification)

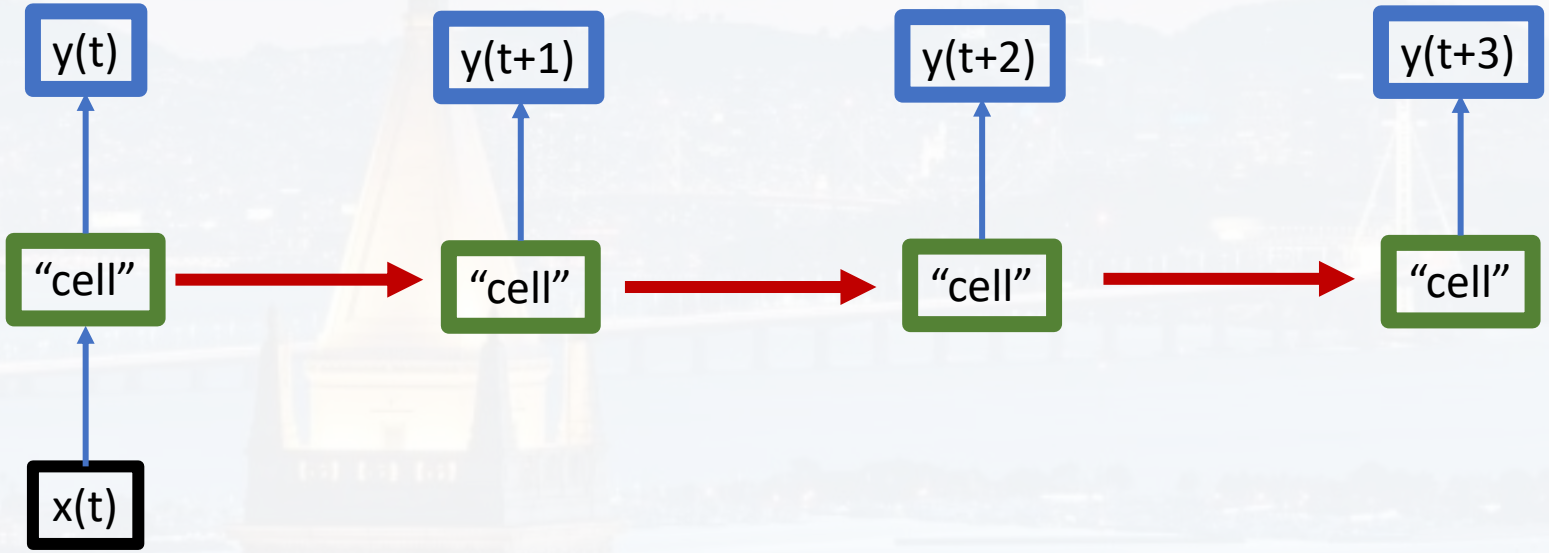




LSTMs:

"one to many"
(image captioning)



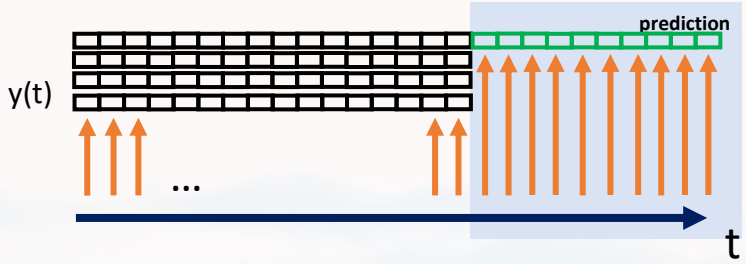


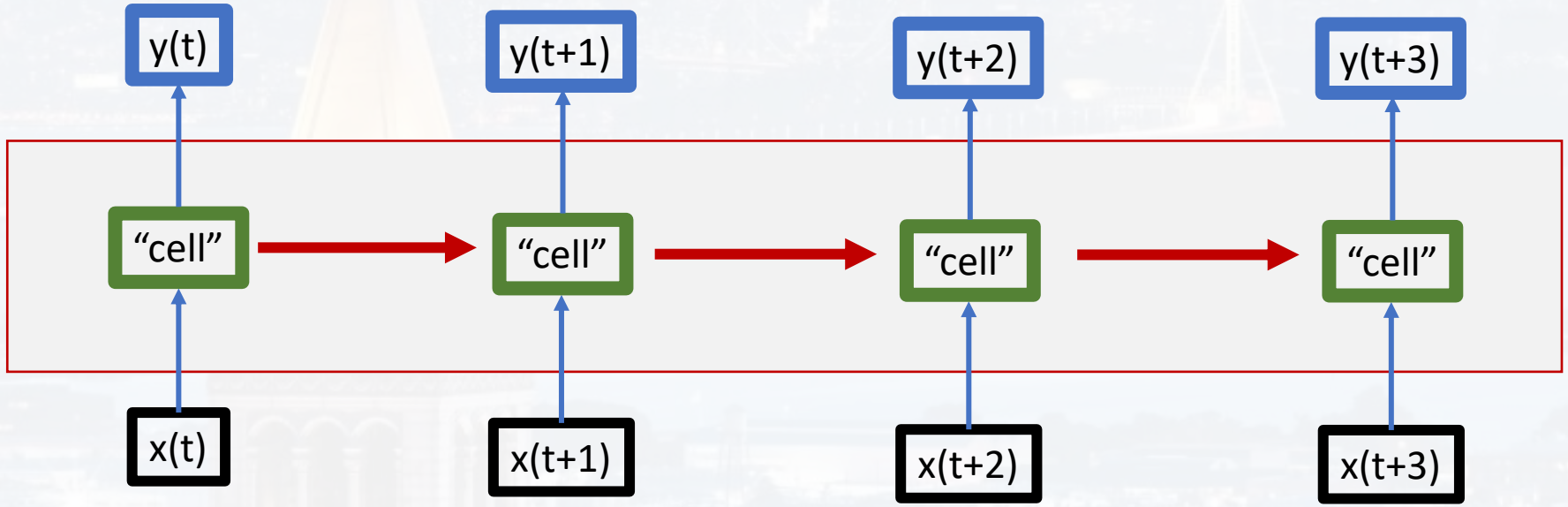
LSTMs:

Applying the **identical** cell **recursively**!

→ easy to implement
→ direction (arrow of time, see later)

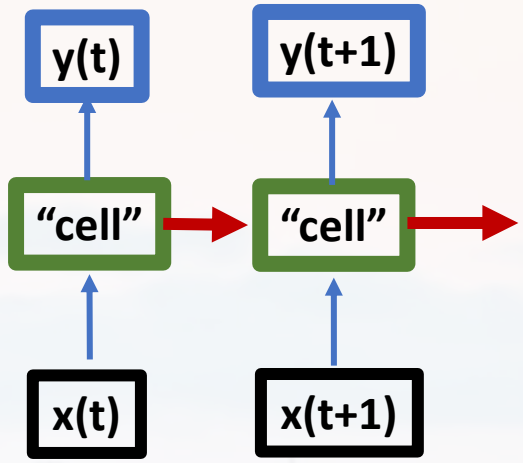
→ exploding/vanishing gradients
→ classic RNN has a "short memory"

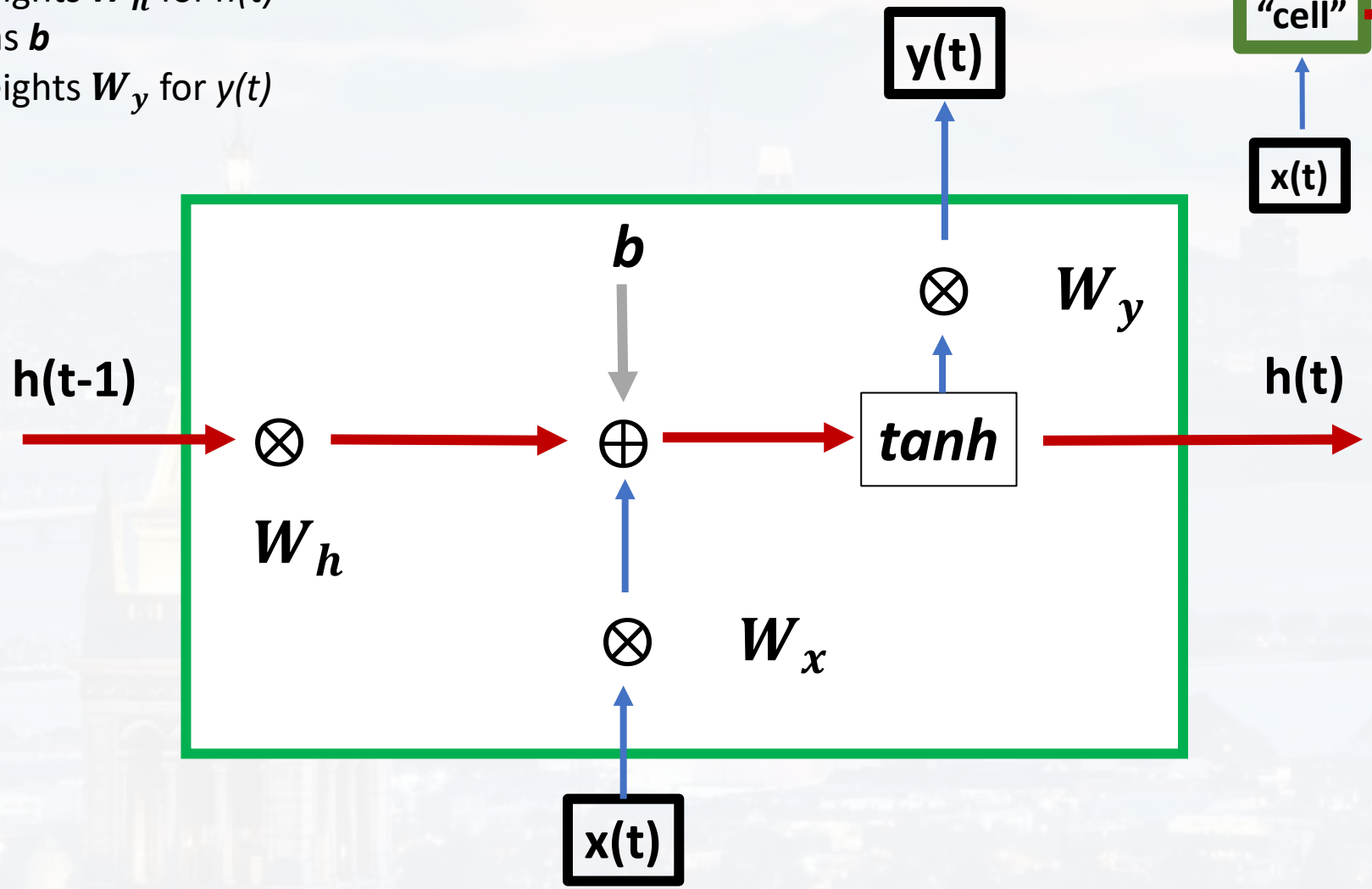




LSTMs:

- **state vector** *h(t)*, i. e. the *"memory"* the system has along time
- weights $\boldsymbol{W_x}$ for *x(t)*
- weights $\boldsymbol{W_h}$ for *h(t)*
- bias ***b***
- weights $\boldsymbol{W_y}$ for *y(t)*

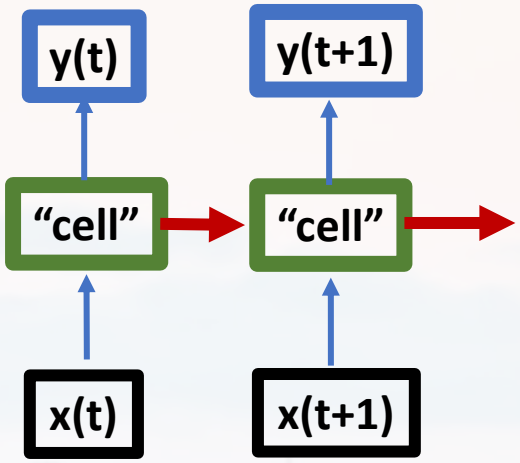


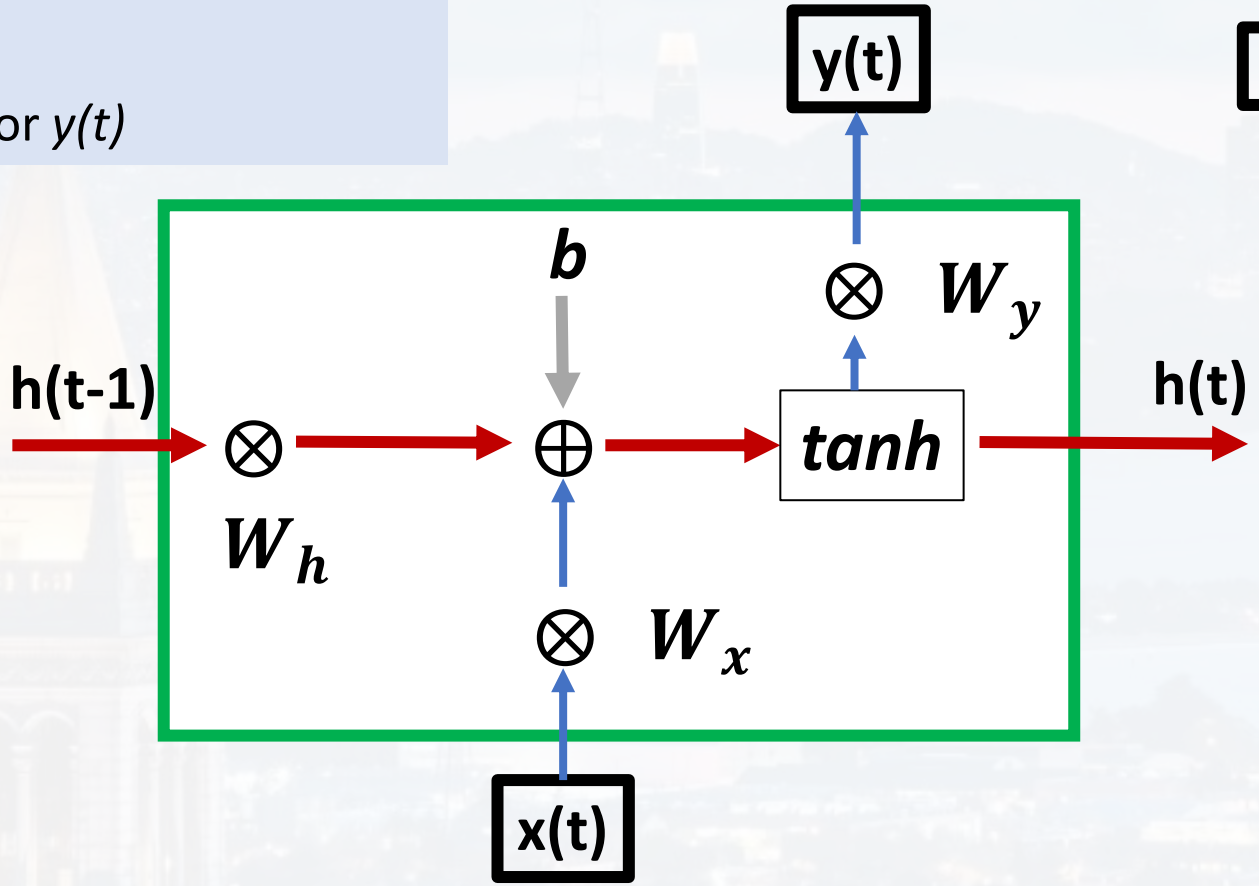


LSTMs:

- **state vector** *h(t)*, i. e. the *"memory"* the system has along time

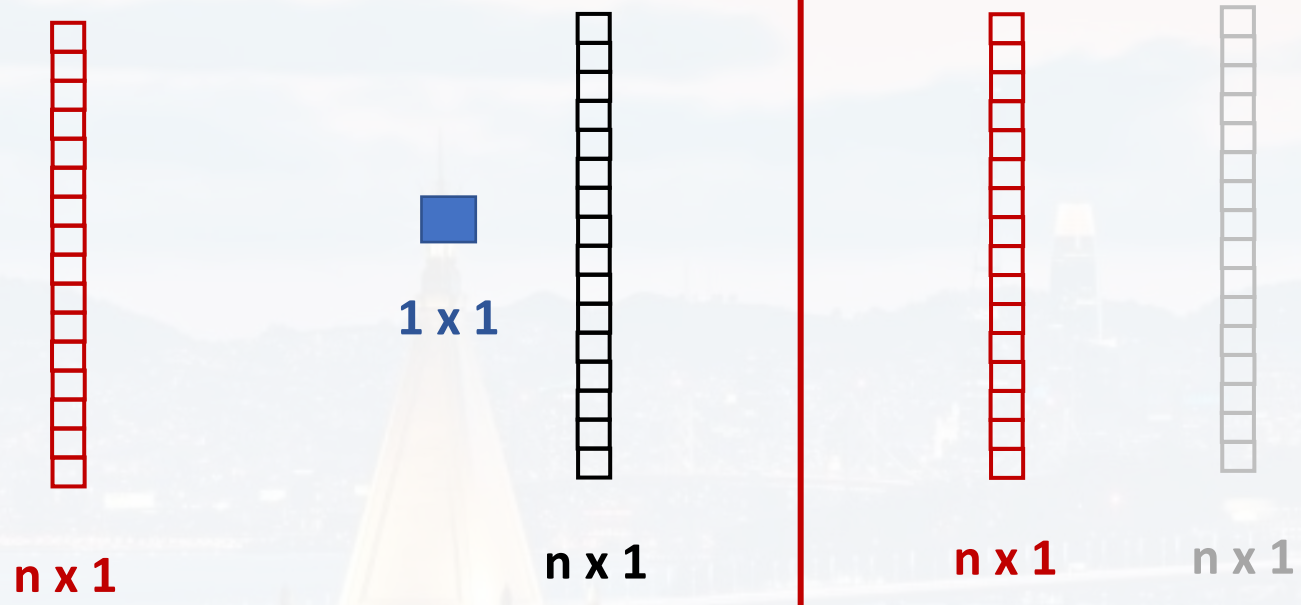
- weights $\boldsymbol{W_x}$ for *x(t)* **trainable**
- weights $\boldsymbol{W_h}$ for *h(t)*
- bias ***b***
- weights $\boldsymbol{W_y}$ for *y(t)*

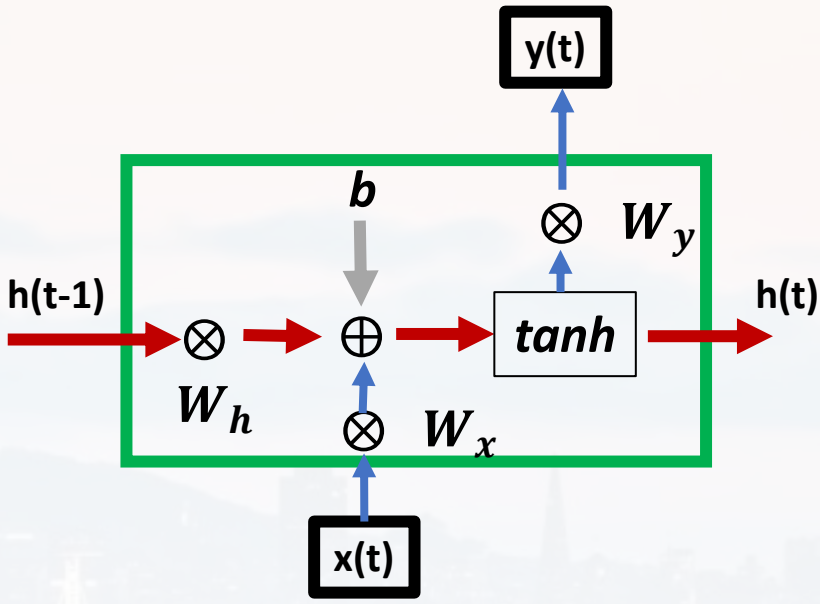




LSTMs:

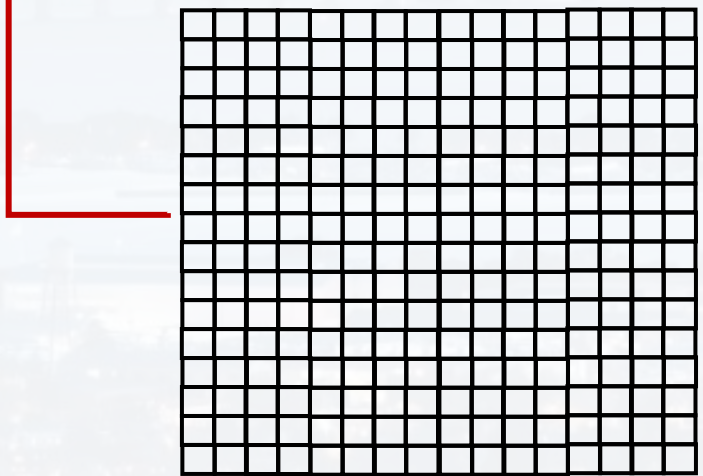
$$h(t) = \tanh[\, x(t) * W_x + \boxed{W_h} * h(t-1) + b \,]$$





n: number of states/ neurons

$$y(t) = h(t) * W_y$$

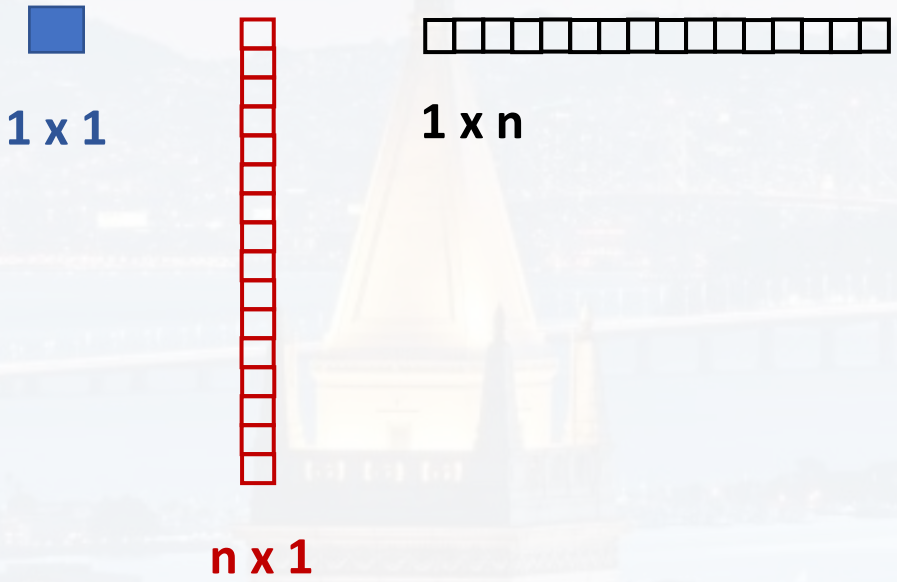


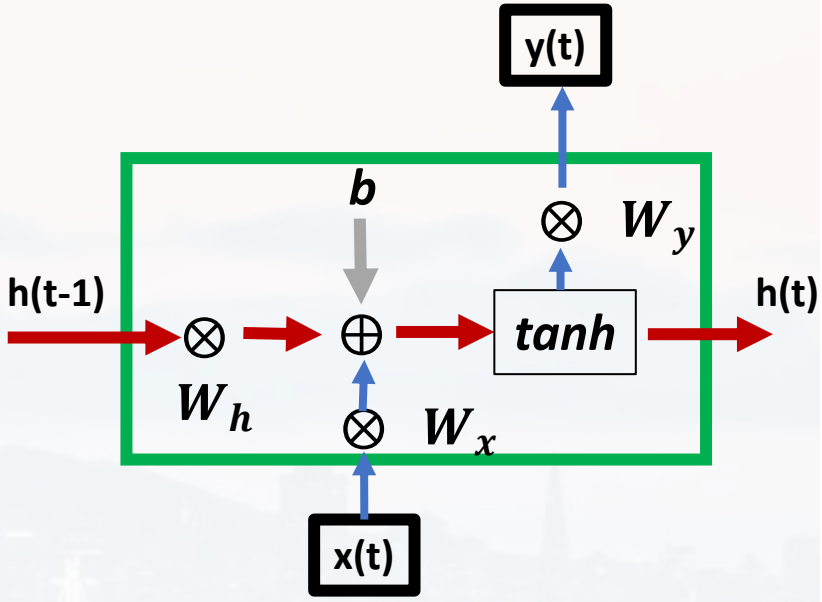
n x n

LSTMs:

$$h(t) = \tanh[\, x(t) * W_x + W_h * h(t-1) + b \,]$$

$$y(t) = h(t) * W_y$$





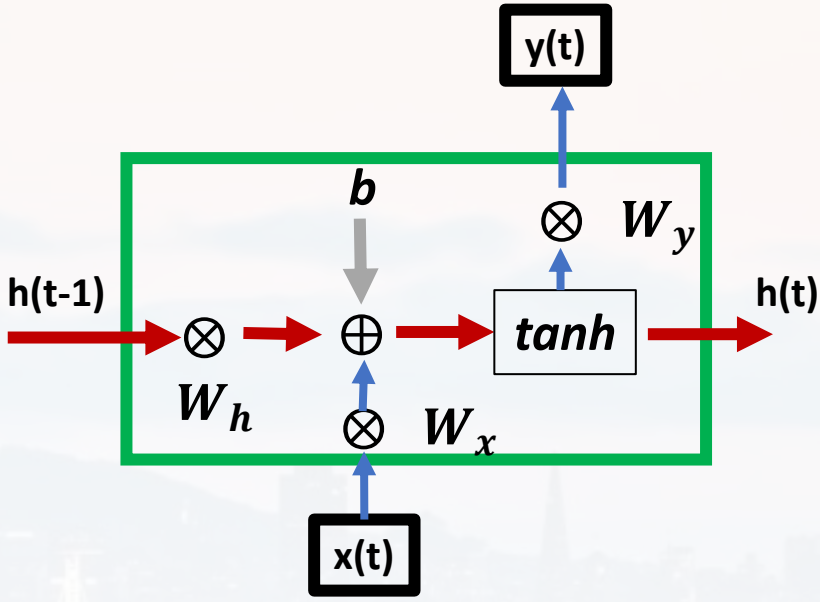
n: number of states/ neurons

LSTMs:

$$h(t) = \tanh[\, x(t) * W_x + W_h * h(t-1) + b \,]$$

$$y(t) = h(t) * W_y$$

usually, $x(t)$ comes in **batches of size B** (= number of sequences) **and of length T** and has **F features** **(see also later)**



n: number of states/ neurons

for **each time point** t:

$$x(t) * W_x^T + h(t-1) * W_h + b$$

for T = 1

shape: $(B \times 1 \times F) * (F \times n)^T + (B \times 1 \times n) * (n \times n) + B \times 1 \times n$

$B \times 1 \times n$ $\quad$ $B \times 1 \times n$

$$y(t) = h(t) * W_y^T$$

shape: $(B \times 1 \times F)$ $\quad$ $(B \times 1 \times n) * (F \times n)^T$

https://www.analyticsvidhya.com

Outline

- Idea and classic RNNs
- **LSTMs**
- *B*ack*P*ropagation *T*hrough *T*ime (BPTT)
- Syntax and some examples

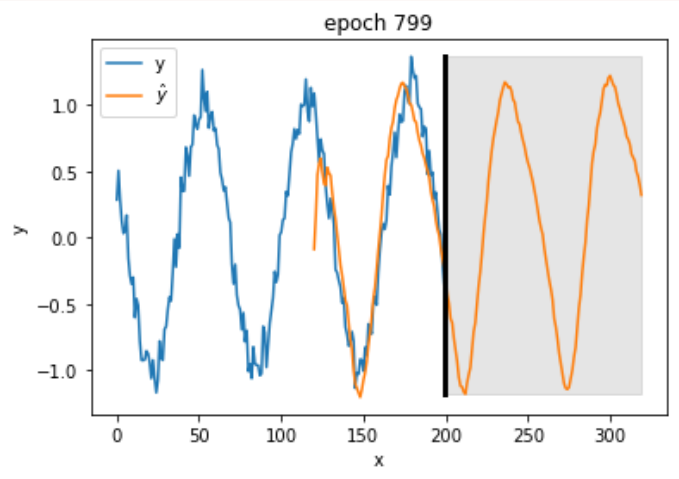
LSTMs:

- **L**ong- **S**hort **T**erm **M**emory

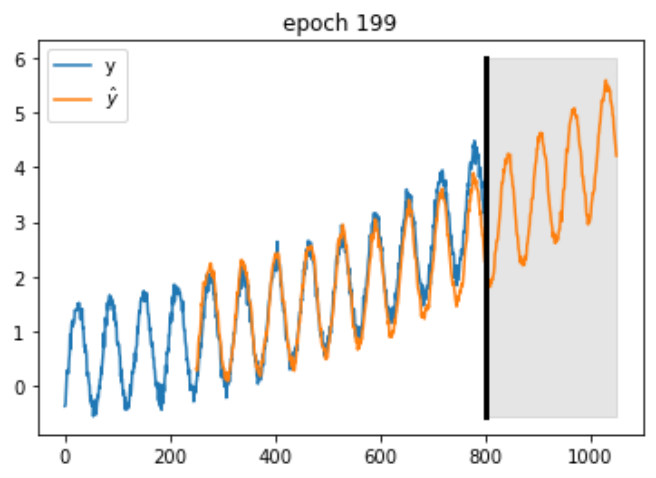
<u>new:</u>

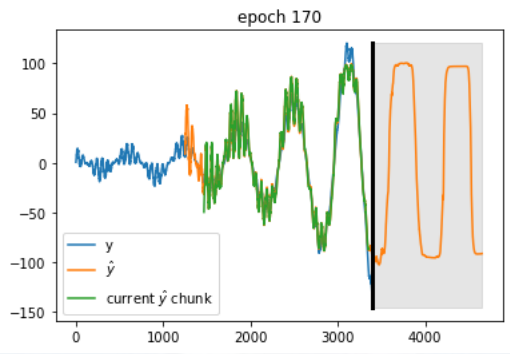
- **long-term** and **short-term** memory
- dealing with vanishing/exploding gradient
- invented 1997 by Sepp Hochreiter und Jürgen Schmidhuber

classical RNN

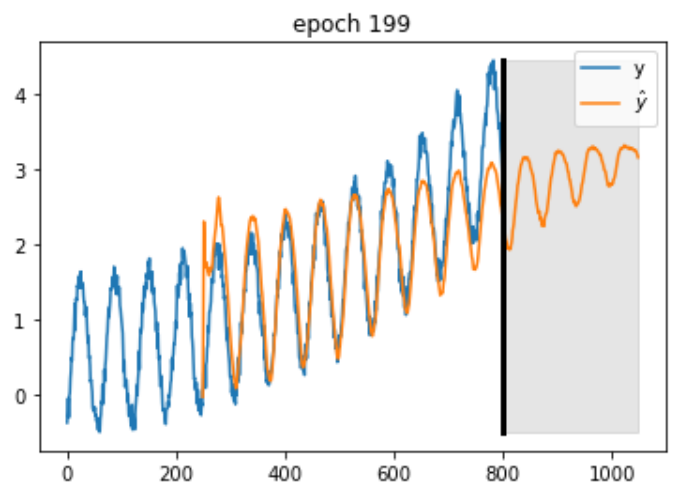


LSTM

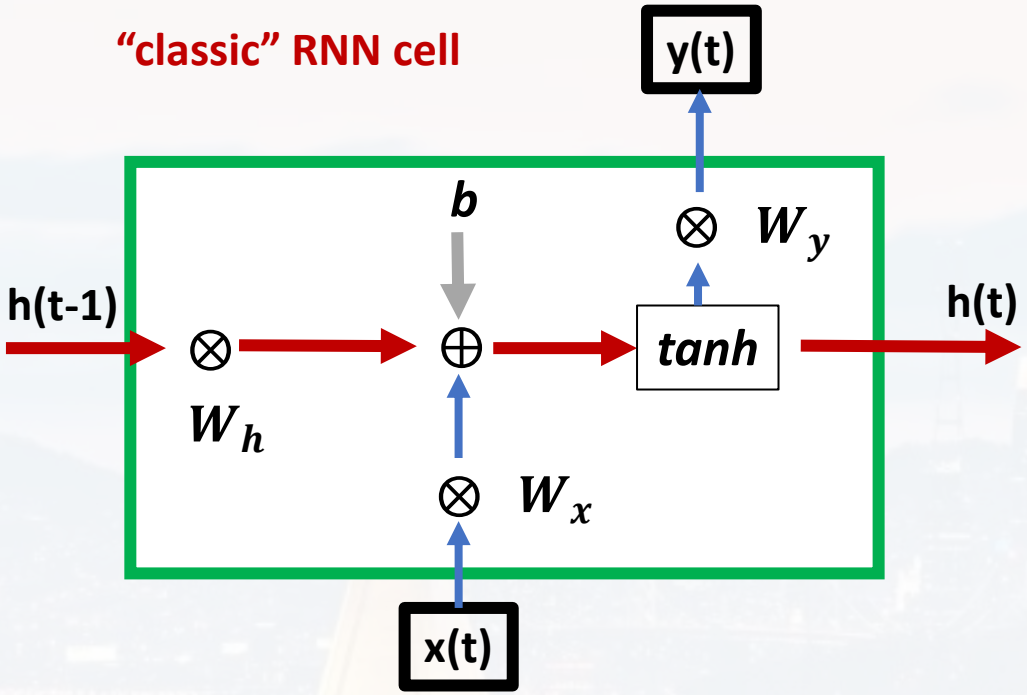


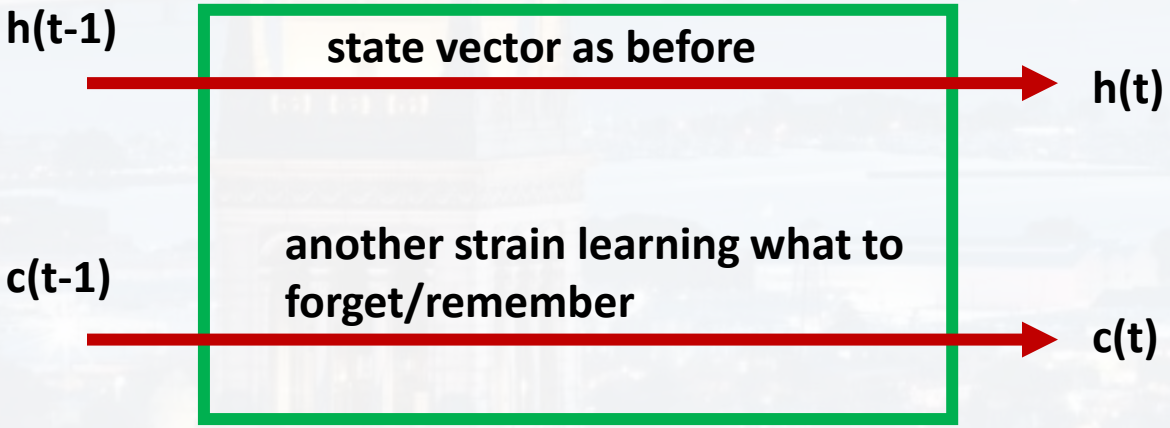


(adding more noise)



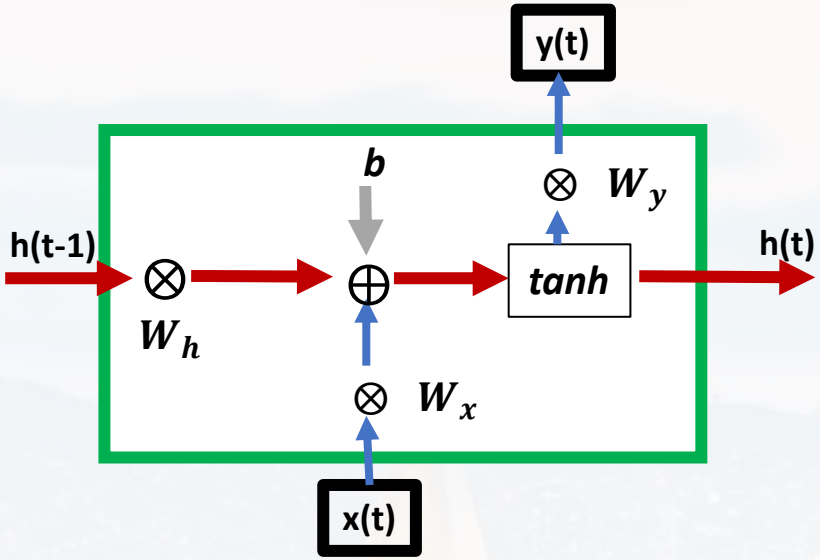
LSTMs:

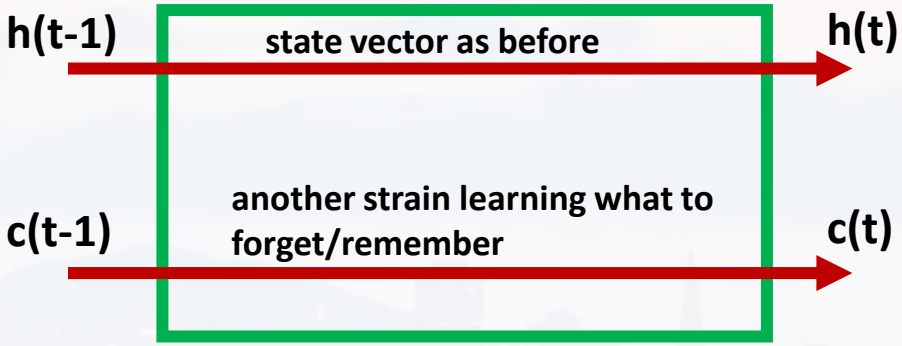

"classic" RNN cell
y(t)
b
W_y
h(t-1)
h(t)
tanh
W_h
W_x
x(t)

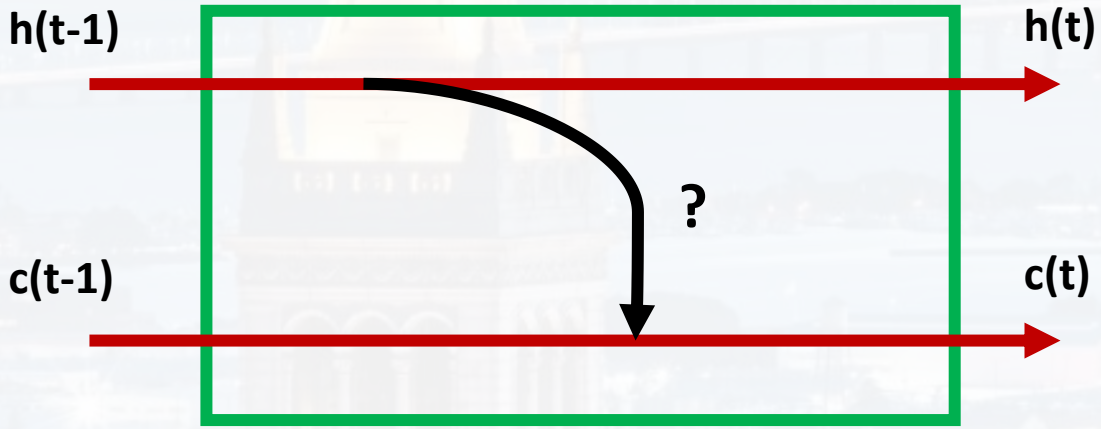

h(t-1)
state vector as before
h(t)
c(t-1)
another strain learning what to forget/remember
c(t)

LSTMs:

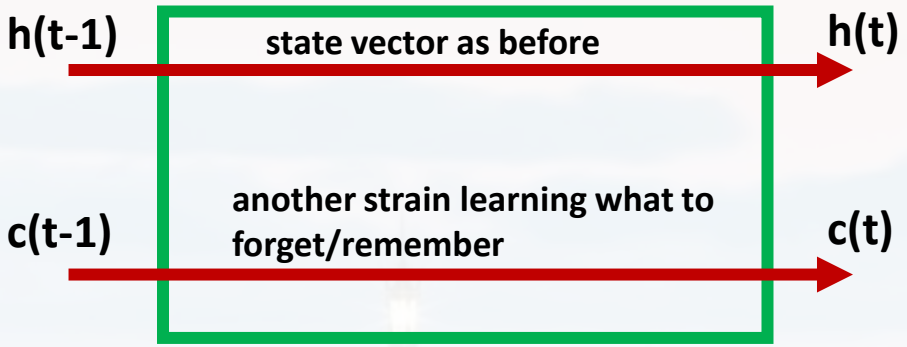
"classic" RNN cell

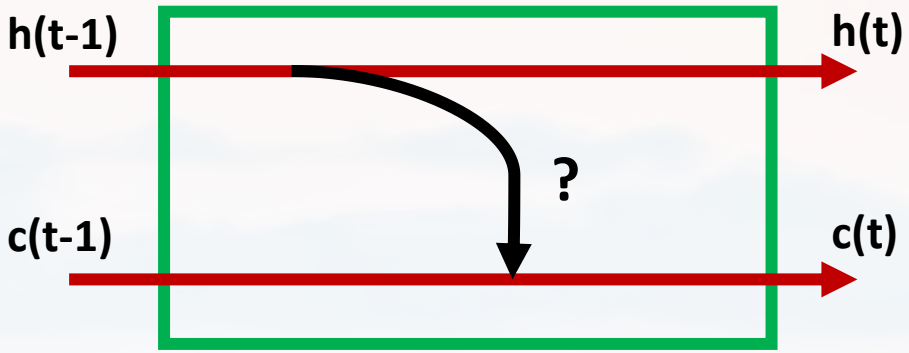




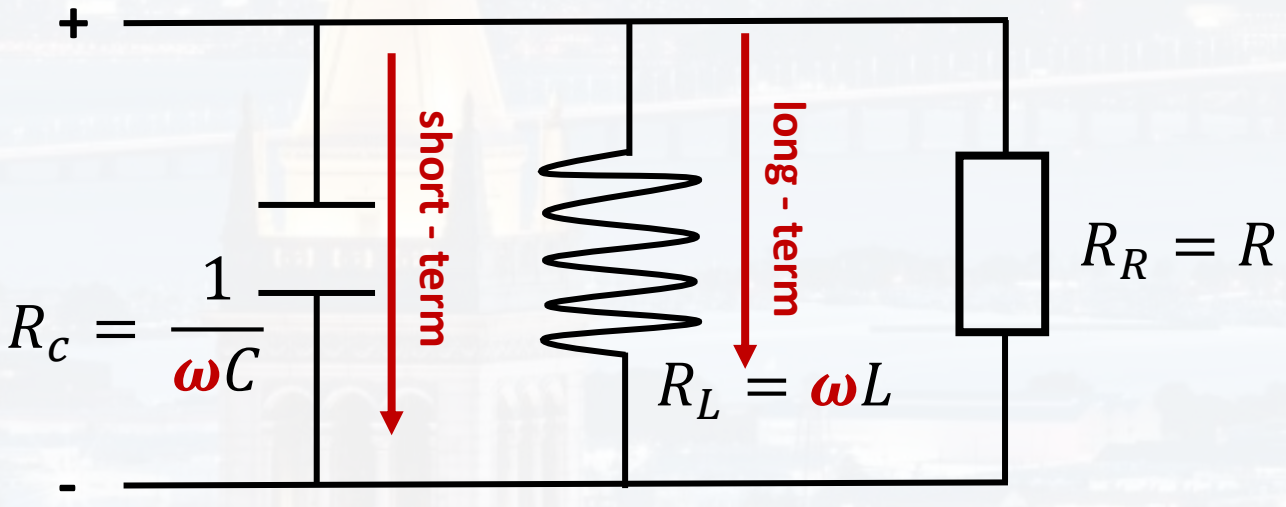


LSTMs:





electrical circuits:



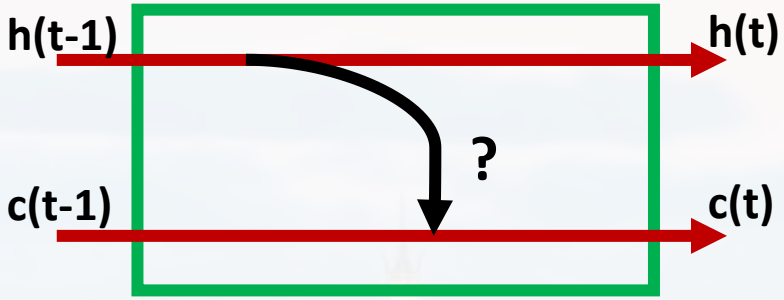
AC: $I(t) = I_0\, e^{i(\omega t + \varphi)}$

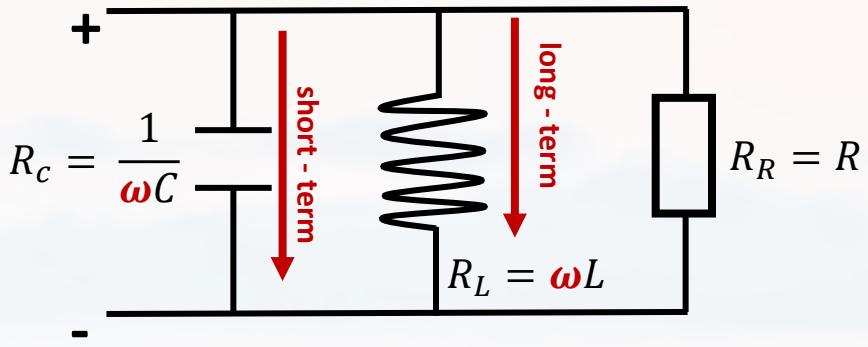
R_c: passes **short** -term changes

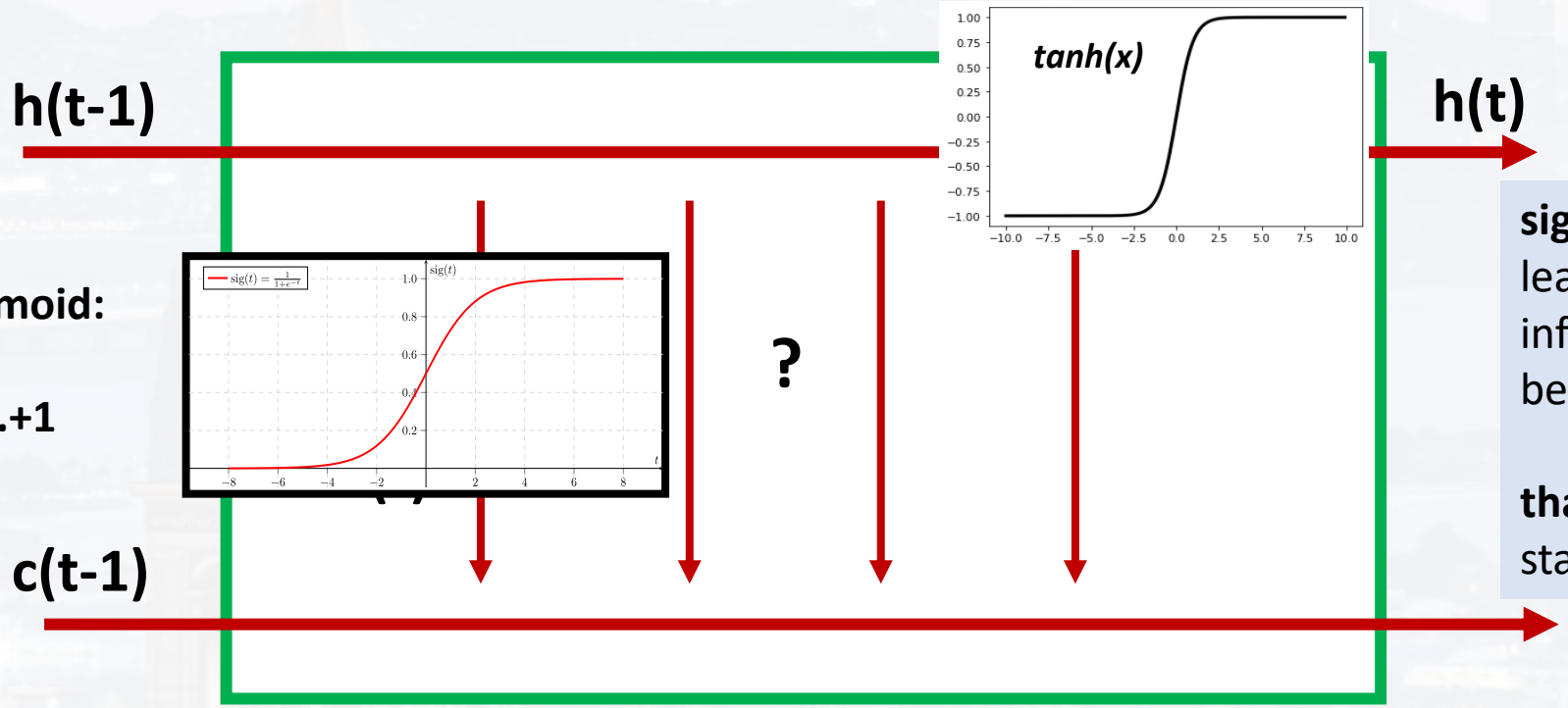
R_L: passes **long** -term changes

$$\frac{1}{R_{tot}} = \frac{1}{R_R} + \frac{1}{R_C} + \frac{1}{R_L}$$

LSTMs:







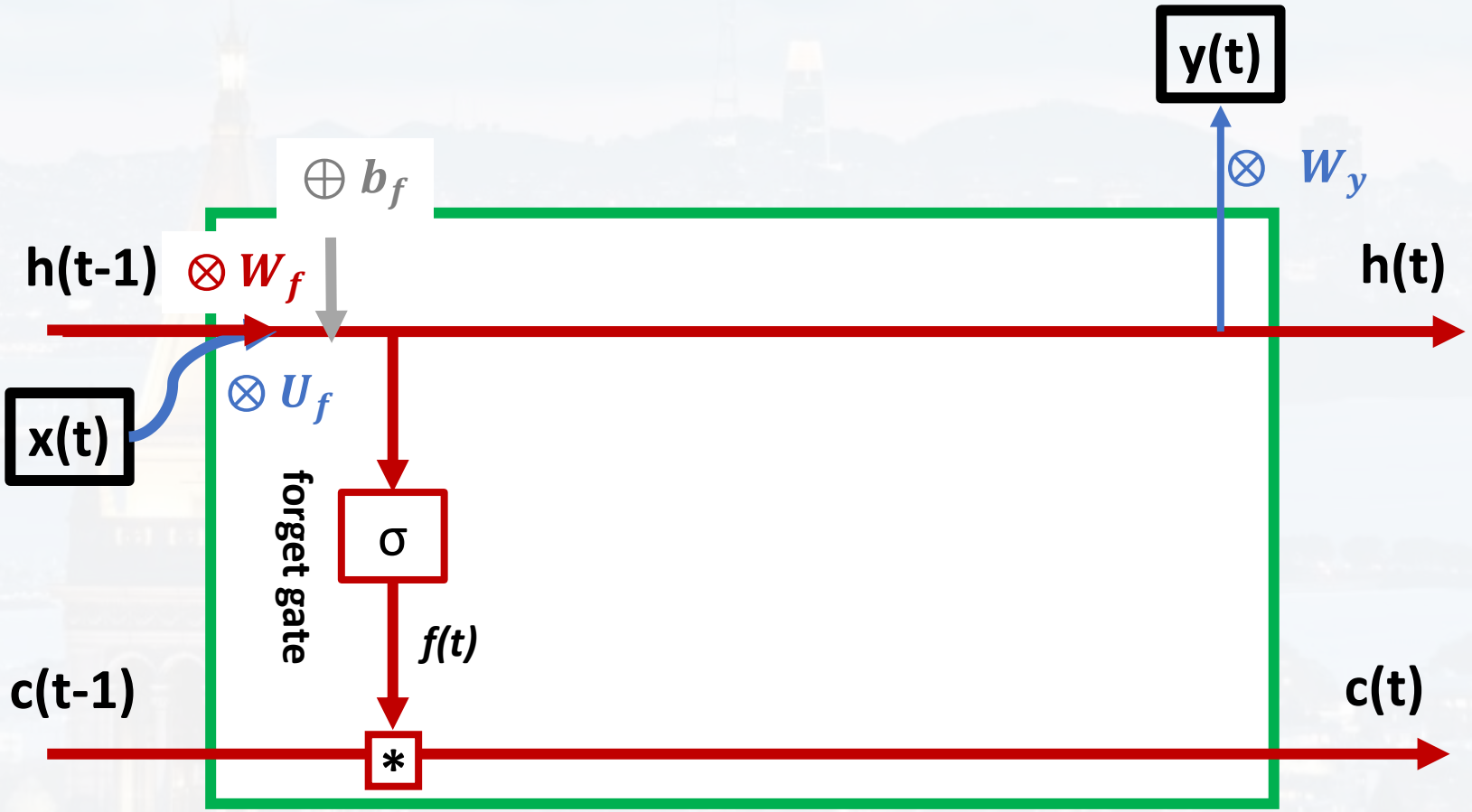
sigmoid:
learning whether information has to be passed on

thanh:
state vector as for RNN

LSTMs:

$$f(t) = \sigma\left(U_f \otimes x(t) + W_f \otimes h(t-1) + b_f\right)$$

* element – wise multiplication



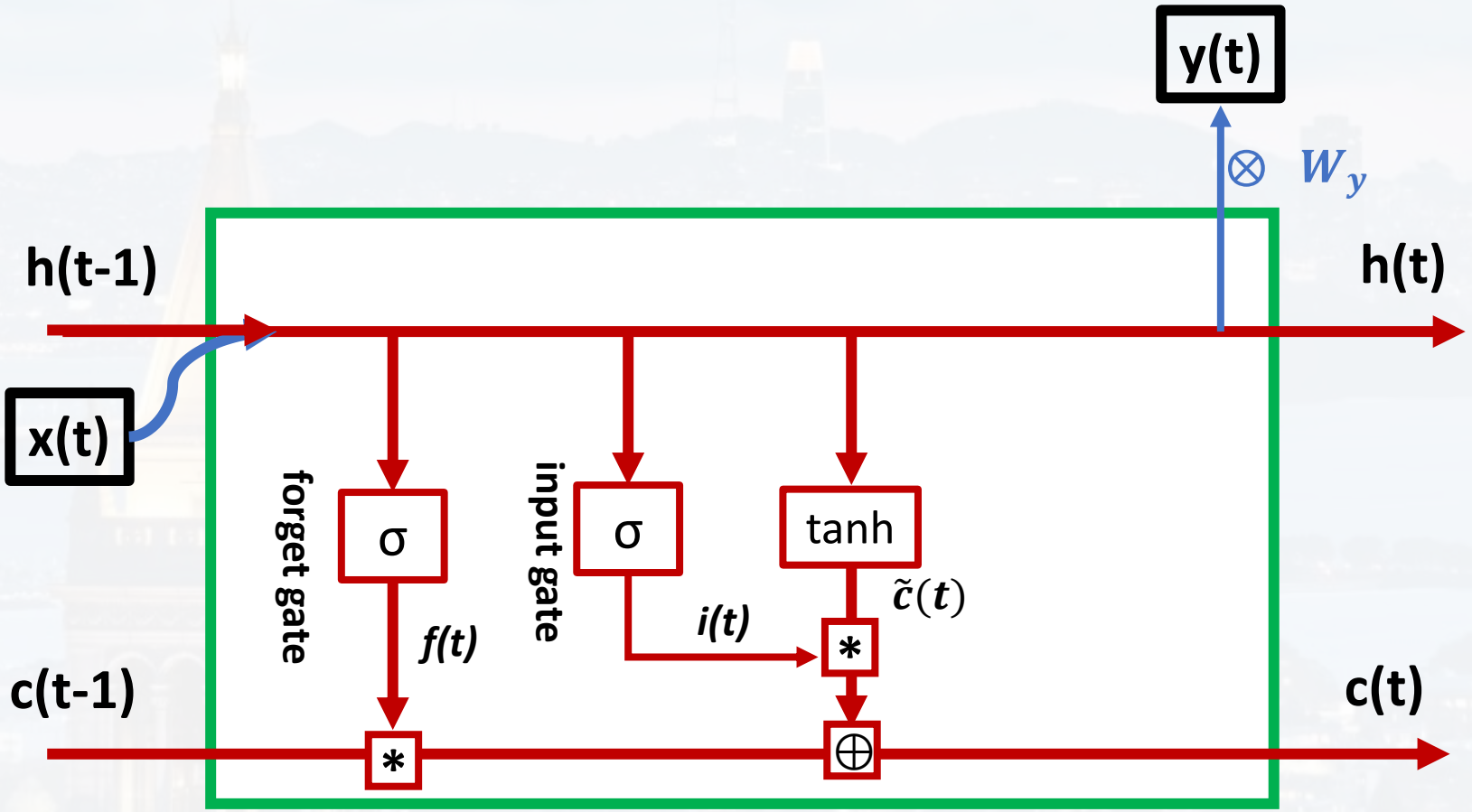
LSTMs:

$$f(t) = \sigma\left(U_f \otimes x(t) + W_f \otimes h(t-1) + b_f\right)$$

$$i(t) = \sigma\left(U_i \otimes x(t) + W_i \otimes h(t-1) + b_i\right)$$

$$\tilde{c}(t) = tanh(U_g \otimes x(t) + W_g \otimes h(t-1) + b_g)$$

* element – wise multiplication



$$c(t) = f(t) * c(t-1) + i(t) * \tilde{c}(t)$$

LSTMs:

$$f(t) = \sigma\left(U_f \otimes x(t) + W_f \otimes h(t-1) + b_f\right)$$

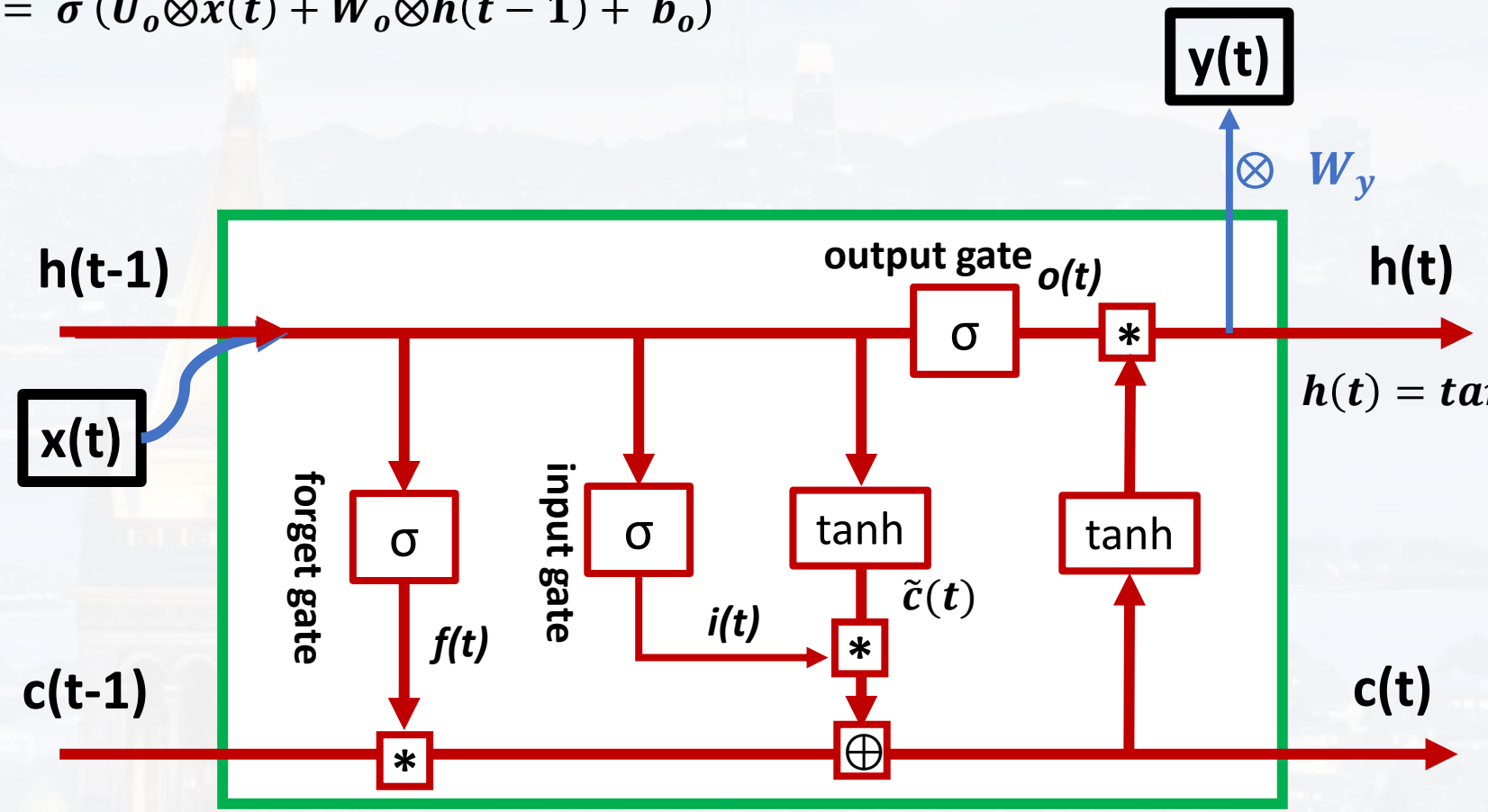
$$i(t) = \sigma\left(U_i \otimes x(t) + W_i \otimes h(t-1) + b_i\right)$$

$$\tilde{c}(t) = tanh(U_g \otimes x(t) + W_g \otimes h(t-1) + b_g)$$

$$c(t) = f(t) * c(t-1) + i(t) * \tilde{c}(t)$$

$$o(t) = \sigma\left(U_o \otimes x(t) + W_o \otimes h(t-1) + b_o\right)$$

* element – wise multiplication



$$h(t) = tanh(c(t)) * o(t)$$

LSTMs:

$$f(t) = \sigma\left(U_f \otimes x(t) + W_f \otimes h(t-1) + b_f\right)$$

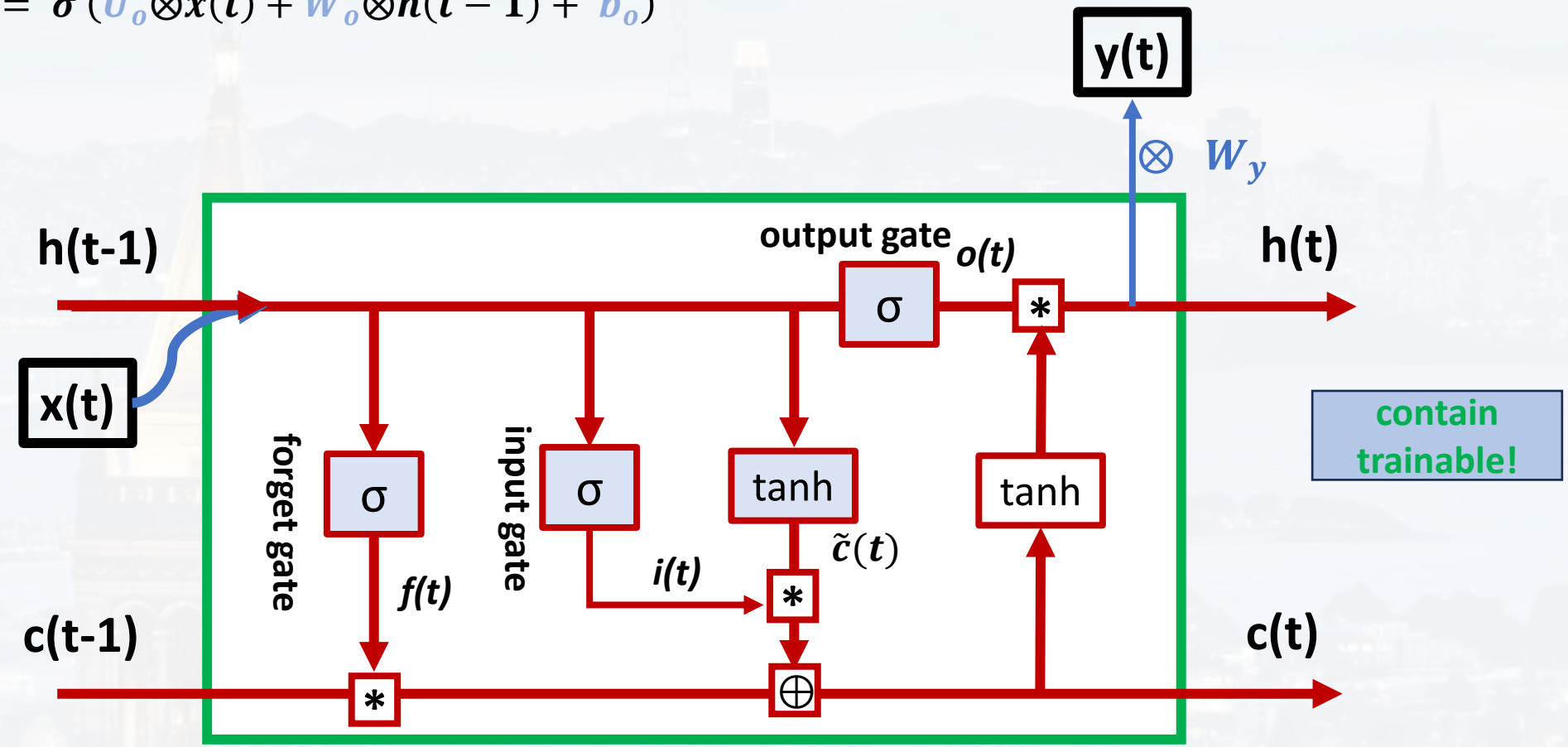
$$i(t) = \sigma\left(U_i \otimes x(t) + W_i \otimes h(t-1) + b_i\right)$$

$$\tilde{c}(t) = tanh(U_g \otimes x(t) + W_g \otimes h(t-1) + b_g)$$

$$c(t) = f(t) * c(t-1) + i(t) * \tilde{c}(t)$$

$$o(t) = \sigma\left(U_o \otimes x(t) + W_o \otimes h(t-1) + b_o\right)$$

* element – wise multiplication

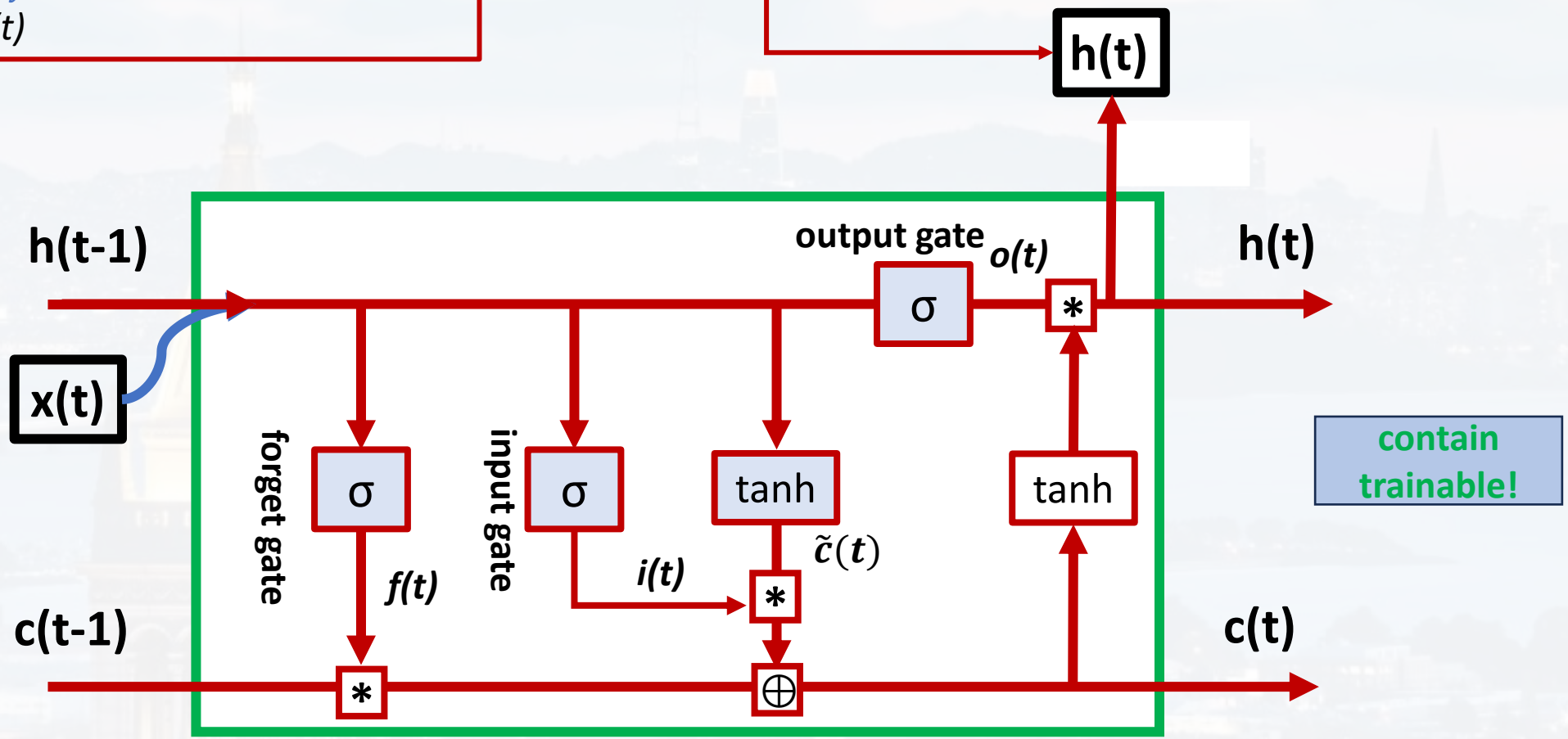


LSTMs:

There is one more thing:

* element – wise multiplication

we will add a **dense layer** instead of W_y at the end to convert *h(t)* to *y(t)*



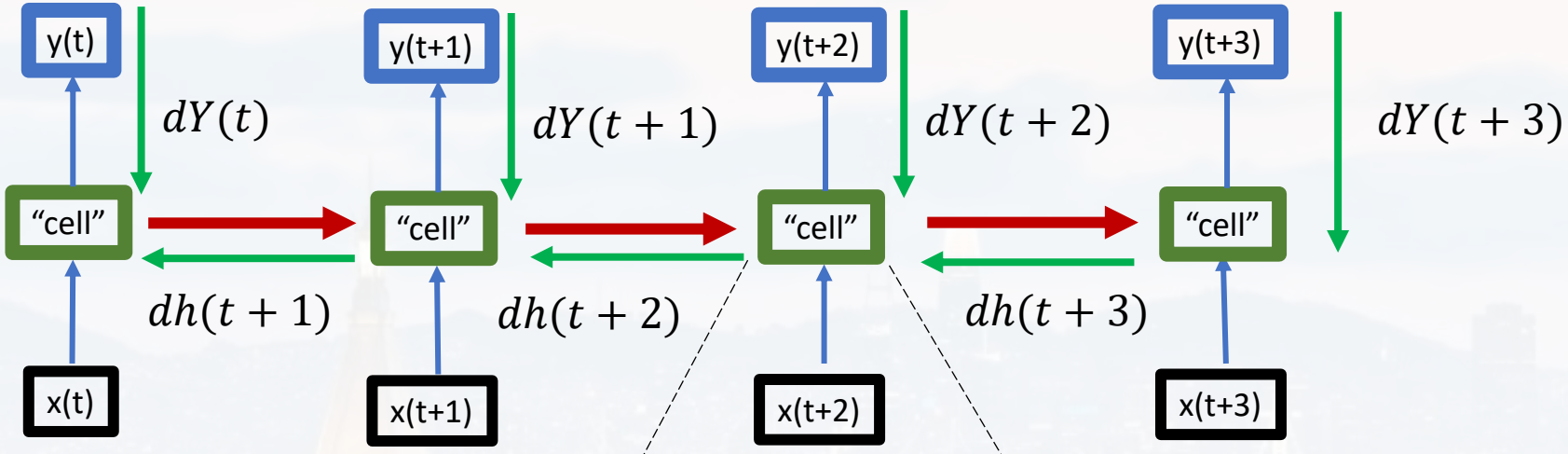
https://www.analyticsvidhya.com

Outline

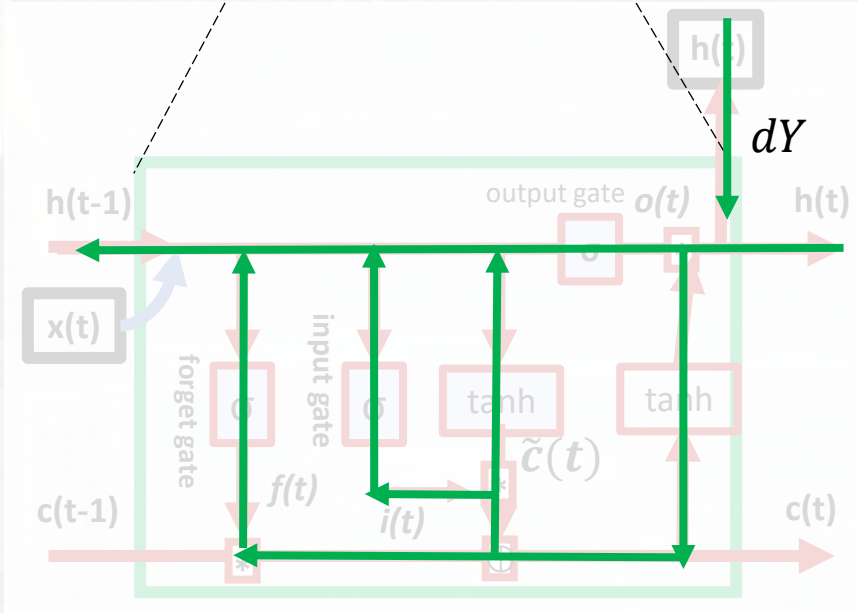
- Idea and classic RNNs
- LSTMs
- **Back*P*ropagation *T*hrough *T*ime (BPTT)**
- Syntax and some examples

LSTMs:

because of the RNN/LSTM architecture, backpropagation works a bit different:

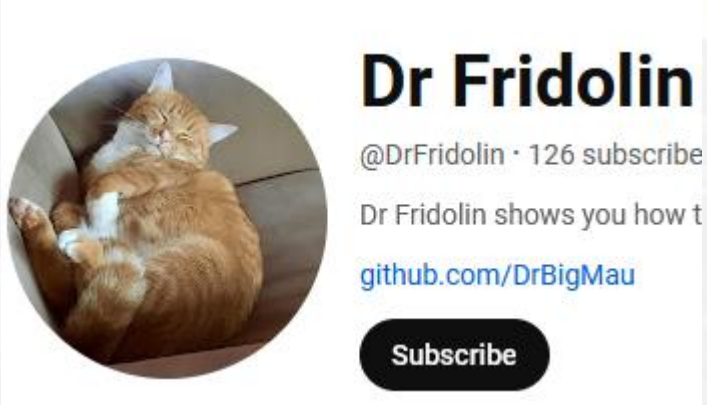


backpropagation through time (BPTT)



note: there is no dc since $c(t)$ is not a trainable!

more details here:



https://www.analyticsvidhya.com

Outline

- Idea and classic RNNs
- LSTMs
- *B*ack*P*ropagation *T*hrough *T*ime (BPTT)
- **Syntax and some examples**

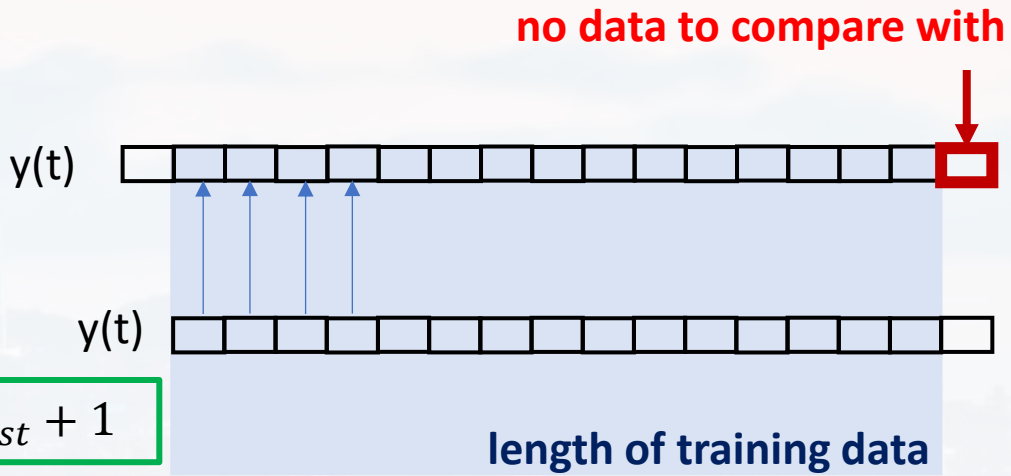
LSTMs:

Let us first understand the logic:

y(t)

predicting **one** step in the future $dt_{futu} = 1$

by **one** step from the past $dt_{past} = 1$



length of training data is: `len[y(t)]` - dt_{futu} - dt_{past} + 1

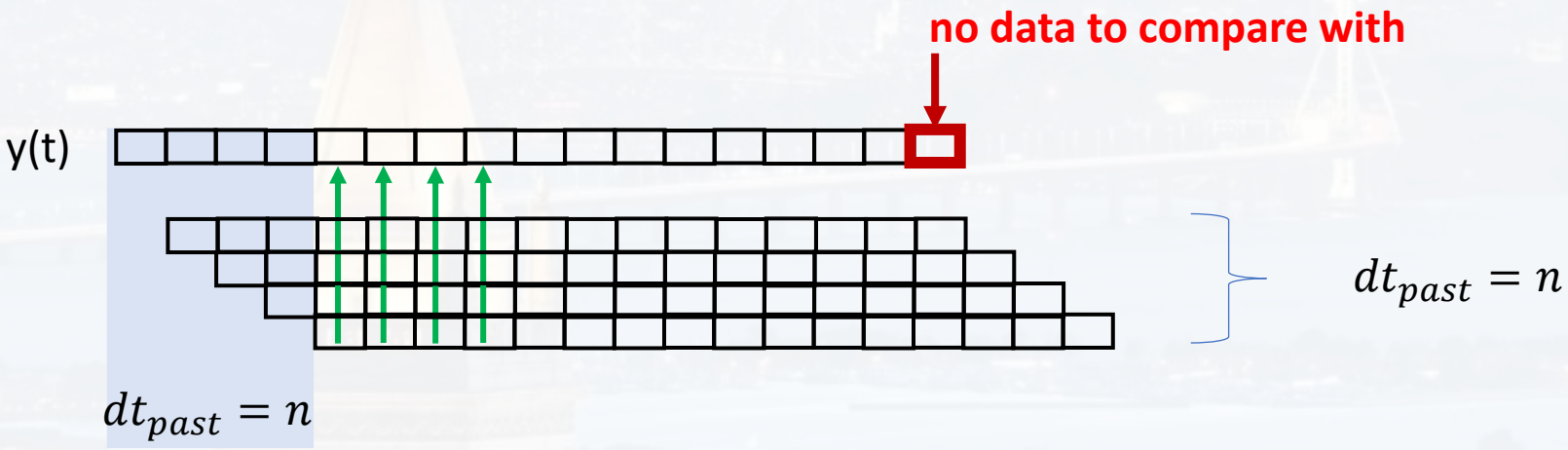
LSTMs:

Let us first understand the logic:

length of training data is: `len[y(t)]` - dt_{futu} - $dt_{past} + 1$

predicting **m** steps in the future $dt_{futu} = m$

by **n** steps from the past $dt_{past} = n$



each of the time points
will influence the forecast

LSTMs:

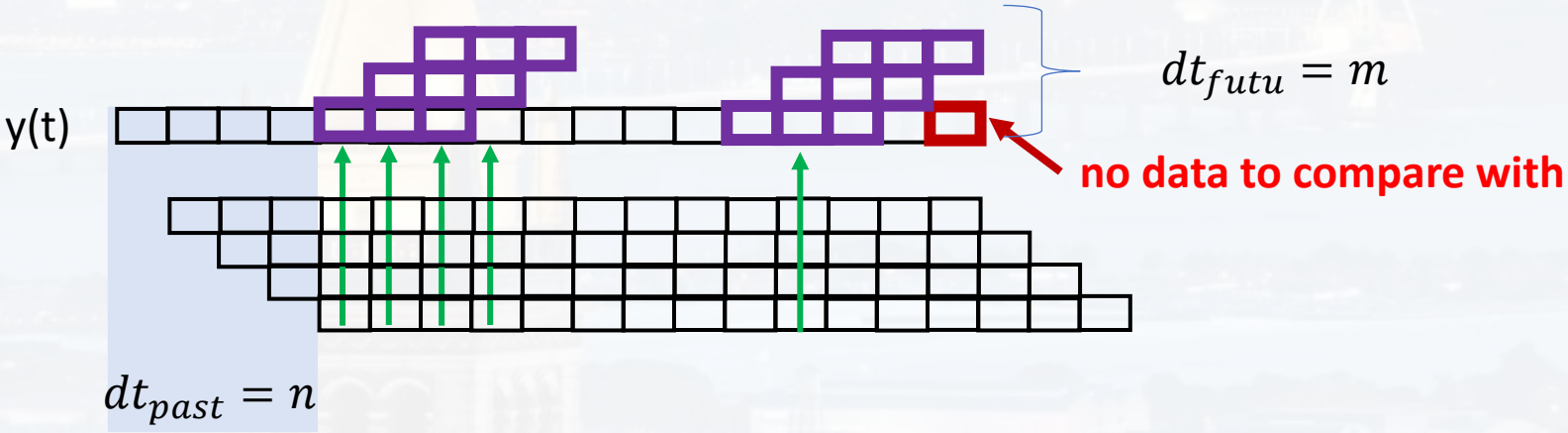
Let us first understand the logic:

length of training data is: `len[y(t)]` - dt_{futu} - $dt_{past} + 1$

predicting **m** steps in the future $dt_{futu} = m$

by **n** steps from the past $dt_{past} = n$

predicting m steps of the future



each of the time points
will influence the forecast

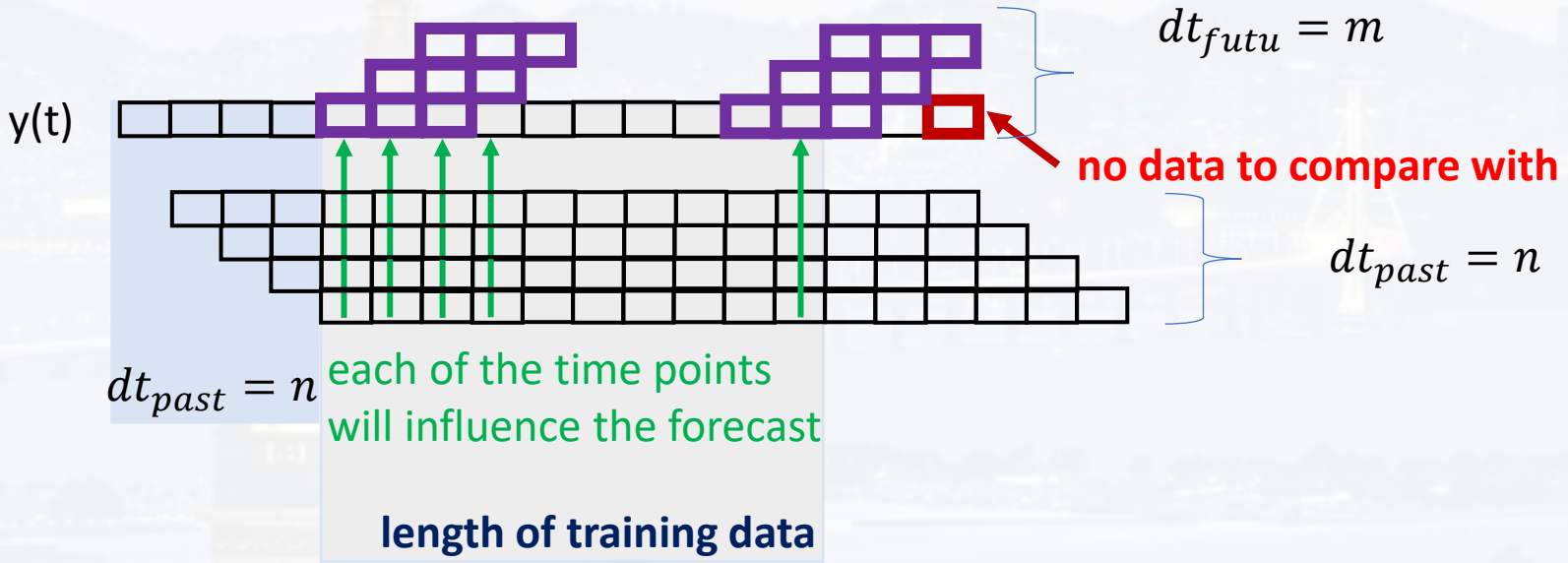
LSTMs:

Let us first understand the logic:

predicting **m** steps in the future $dt_{futu} = m$

by **n** steps from the past $dt_{past} = n$

predicting m steps of the future



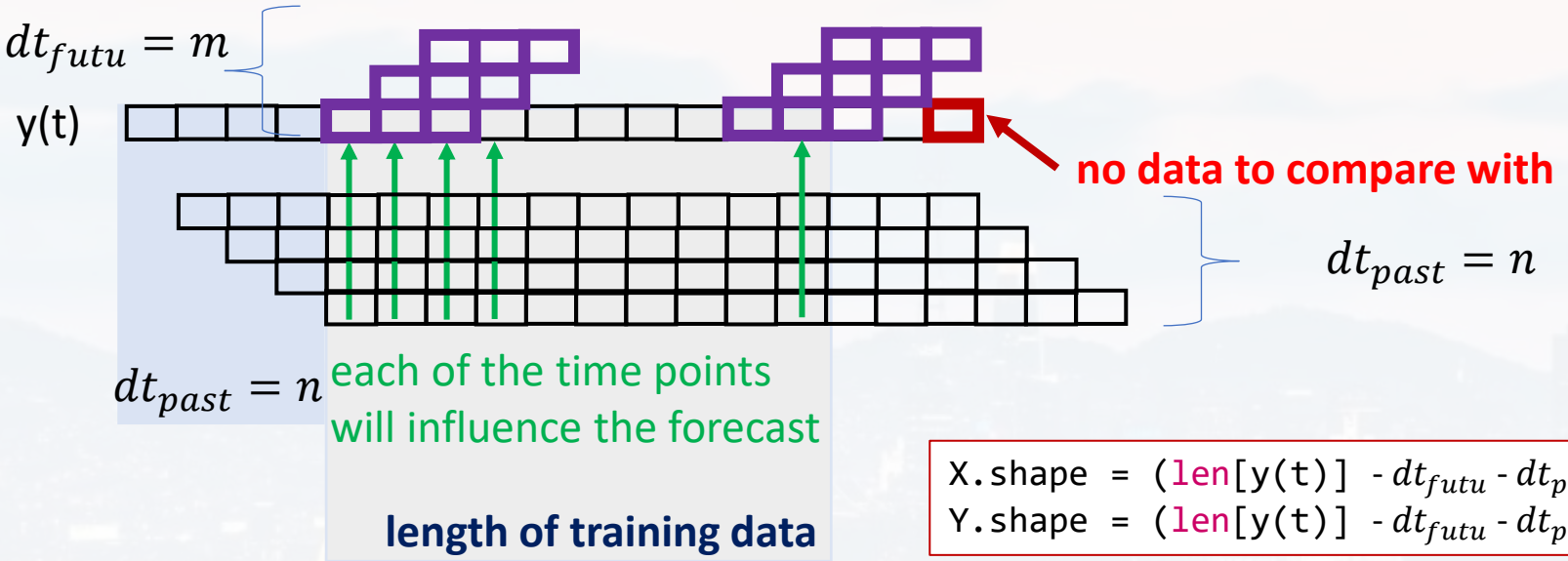

```
X.shape = (len[y(t)] - dt_futu - dt_past + 1) x dt_past x n_feature(X)
Y.shape = (len[y(t)] - dt_futu - dt_past + 1) x dt_futu x n_feature(Y)
```

LSTMs: Syntax and some examples

predicting m steps of the future

predicting **m** steps in the future $dt_{futu} = m$

by **n** steps from the past $dt_{past} = n$




```
X.shape = (len[y(t)] - dt_futu - dt_past + 1) x dt_past x n_feature(X)
Y.shape = (len[y(t)] - dt_futu - dt_past + 1) x dt_futu x n_feature(Y)
```

X dt_{past}

len[y(t)] - dt_{futu} - dt_{past} + 1

```
[0.23364871, 0.25531086, 0.29226308, 0.30477917, 0.34526381]
[0.25531086, 0.29226308, 0.30477917, 0.34526381, 0.32876229]
[0.29226308, 0.30477917, 0.34526381, 0.32876229, 0.34967038]
[0.30477917, 0.34526381, 0.32876229, 0.34967038, 0.32374534]
[0.34526381, 0.32876229, 0.34967038, 0.32374534, 0.34168462]
[0.32876229, 0.34967038, 0.32374534, 0.34168462, 0.27602807]
[0.34967038, 0.32374534, 0.34168462, 0.27602807, 0.2313527 ]
[0.32374534, 0.34168462, 0.27602807, 0.2313527 , 0.20877584]
[0.34168462, 0.27602807, 0.2313527 , 0.20877584, 0.16455034]
[0.27602807, 0.2313527 , 0.20877584, 0.16455034, 0.11714726]
```

Y dt_{futu}

```
[0.05263142, 0.10779498, 0.12263184],
[0.10779498, 0.12263184, 0.12821065],
[0.12263184, 0.12821065, 0.20806335],
[0.12821065, 0.20806335, 0.2518744 ],
[0.20806335, 0.2518744 , 0.28025766],
[0.2518744 , 0.28025766, 0.27699119],
[0.28025766, 0.27699119, 0.30965494],
[0.27699119, 0.30965494, 0.37666627],
[0.30965494, 0.37666627, 0.37879347],
[0.37666627, 0.37879347, 0.36811853]]
```

LSTMs:

Let us first understand the logic:

Once, we have fitted the model: how do we apply the prediction?

```
PredY = model.predict(TestX)

(TestX.shape[0], dt_futu) = PredY.shape
```

X dt_{past}

len[y(t)] - dt_{futu} - dt_{past} + 1

```
[0.23364871, 0.25531086, 0.29226308, 0.30477917, 0.34526381]
[0.25531086, 0.29226308, 0.30477917, 0.34526381, 0.32876229]
[0.29226308, 0.30477917, 0.34526381, 0.32876229, 0.34967038]
[0.30477917, 0.34526381, 0.32876229, 0.34967038, 0.32374534]
[0.34526381, 0.32876229, 0.34967038, 0.32374534, 0.34168462]
[0.32876229, 0.34967038, 0.32374534, 0.34168462, 0.27602807]
[0.34967038, 0.32374534, 0.34168462, 0.27602807, 0.2313527 ]
[0.32374534, 0.34168462, 0.27602807, 0.2313527 , 0.20877584]
[0.34168462, 0.27602807, 0.2313527 , 0.20877584, 0.16455034]
[0.27602807, 0.2313527 , 0.20877584, 0.16455034, 0.11714726]
```

Y dt_{futu}

```
[0.05263142, 0.10779498, 0.12263184],
[0.10779498, 0.12263184, 0.12821065],
[0.12263184, 0.12821065, 0.20806335],
[0.12821065, 0.20806335, 0.2518744 ],
[0.20806335, 0.2518744 , 0.28025766],
[0.2518744 , 0.28025766, 0.27699119],
[0.28025766, 0.27699119, 0.30965494],
[0.27699119, 0.30965494, 0.37666627],
[0.30965494, 0.37666627, 0.37879347],
[0.37666627, 0.37879347, 0.36811853]]
```

`TestX[0,:,0]` should predict `TestY[0,:,0]`
`TestX[1,:,0]` should predict `TestY[1,:,0]` etc

LSTMs:

Syntax and some examples

Let us explore `LSTMI.ipynb`

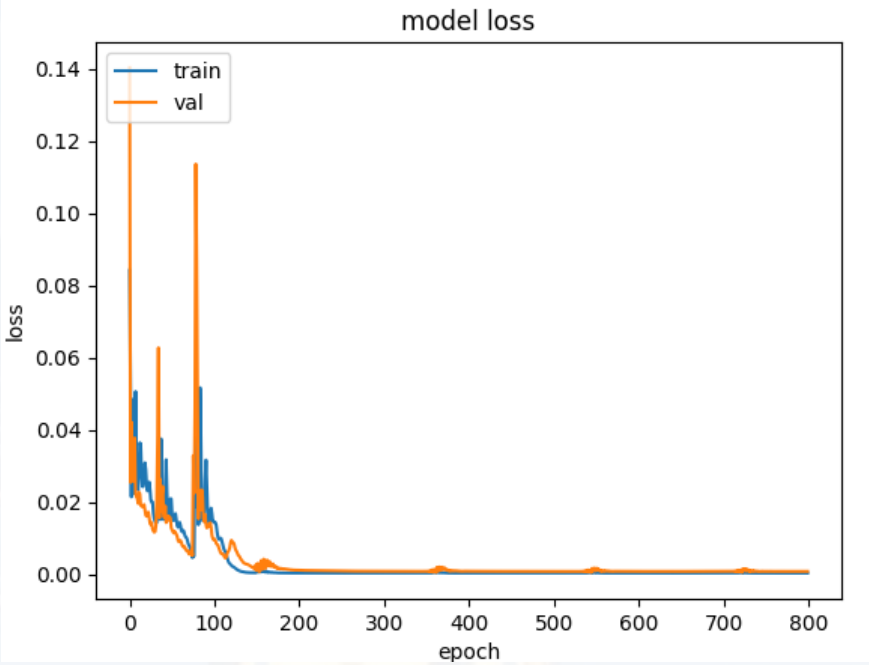
```
n_neurons  = 400
batch_size = 128

model = Sequential()
model.add(LSTM(n_neurons, input_shape = (dt_past, n_features),\
               activation = 'tanh'))
model.add(Dense(dt_futu))

opt = optimizers.Adam()
model.compile(loss = 'mean_squared_error', optimizer = opt)

model.summary()
```

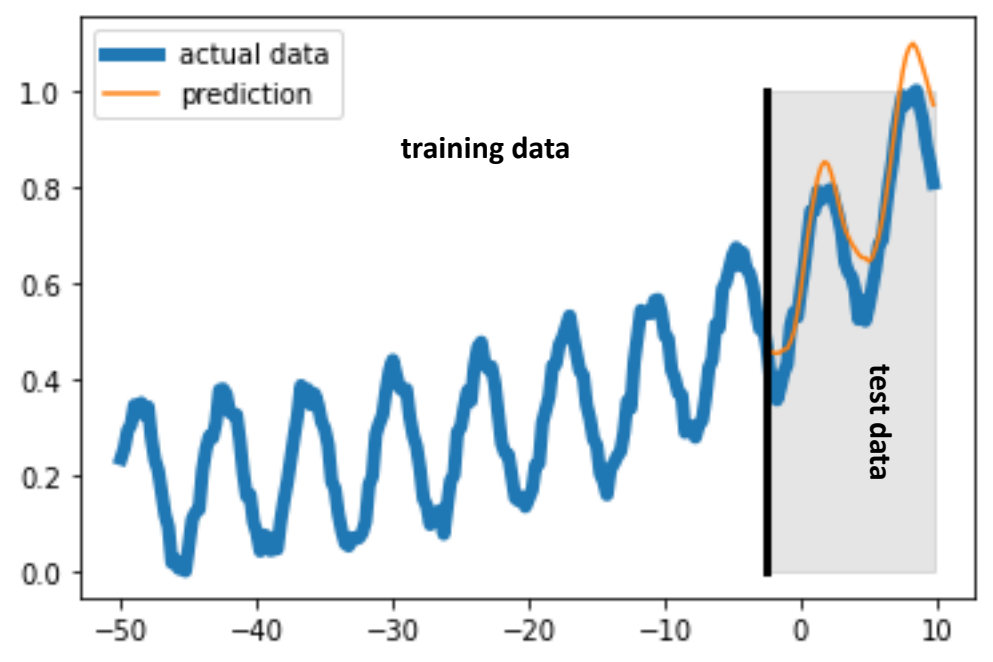
```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 lstm (LSTM)                 (None, 400)               643200

 dense (Dense)               (None, 8)                 3208

=================================================================
Total params: 646408 (2.47 MB)
Trainable params: 646408 (2.47 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

LSTMs:

Syntax and some examples

Let us explore `LSTMI.ipynb`

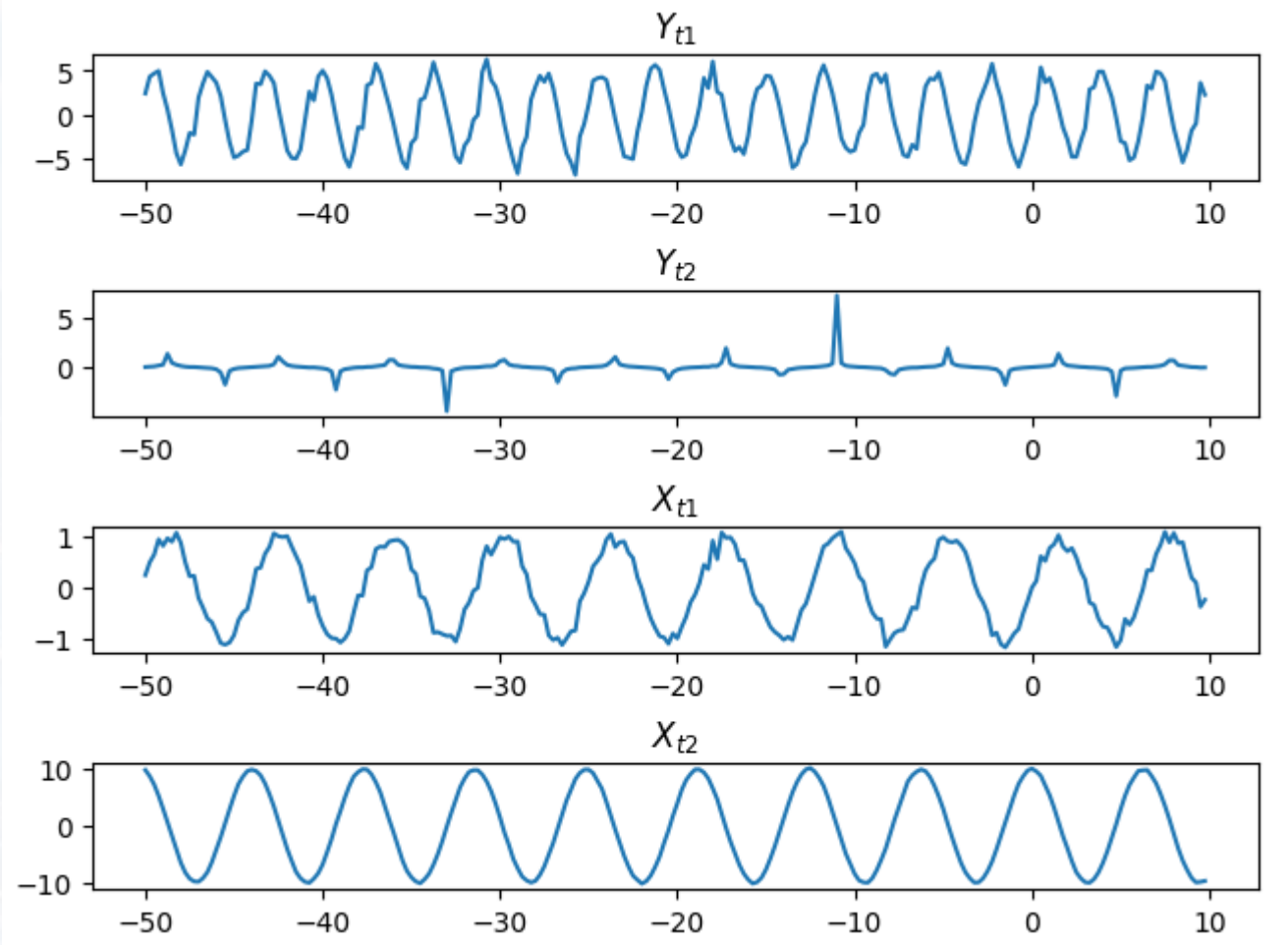




LSTMs:

Syntax and some examples

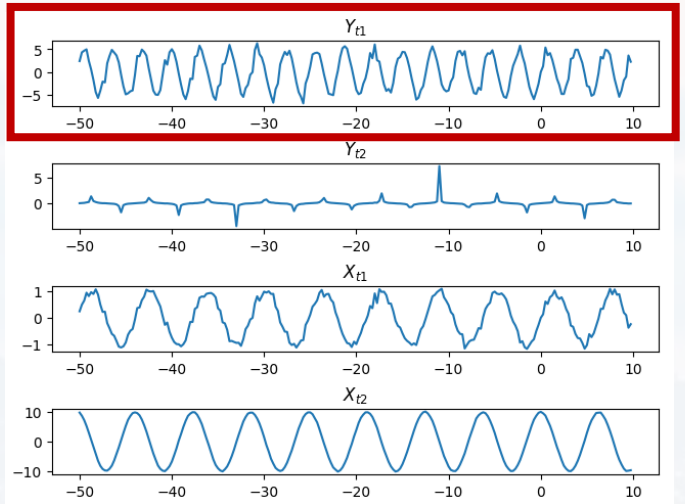
Explore `LSTMII.ipynb` for a multivariate, multi feature time series:

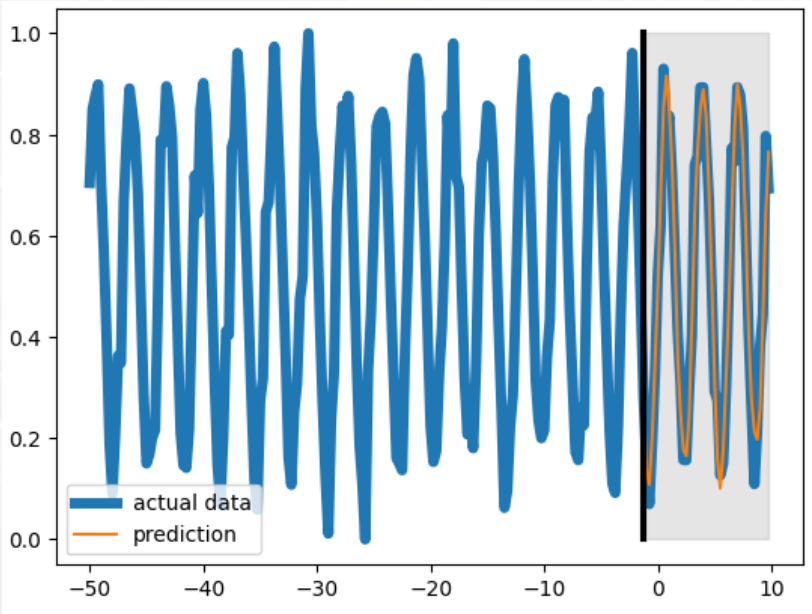


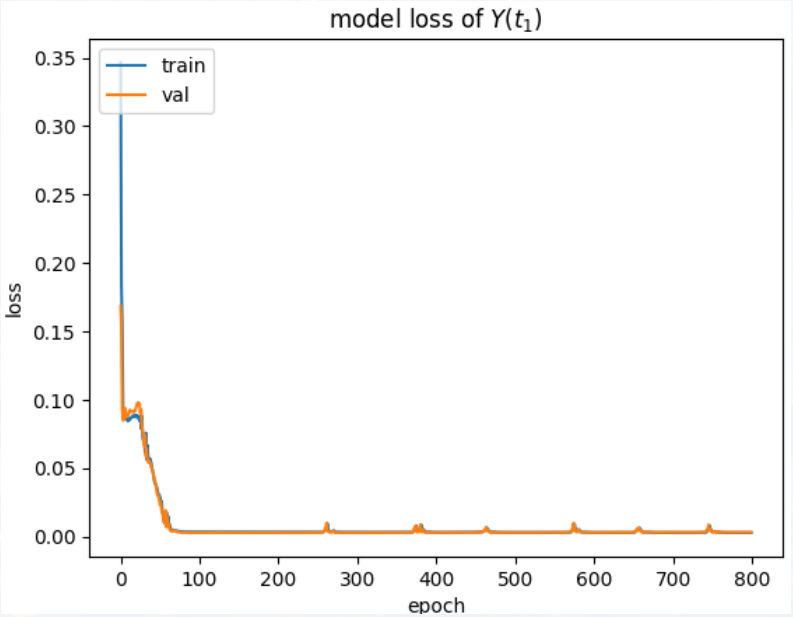
LSTMs:

Syntax and some examples

Explore `LSTMII.ipynb` for a multivariate, multi feature time series:



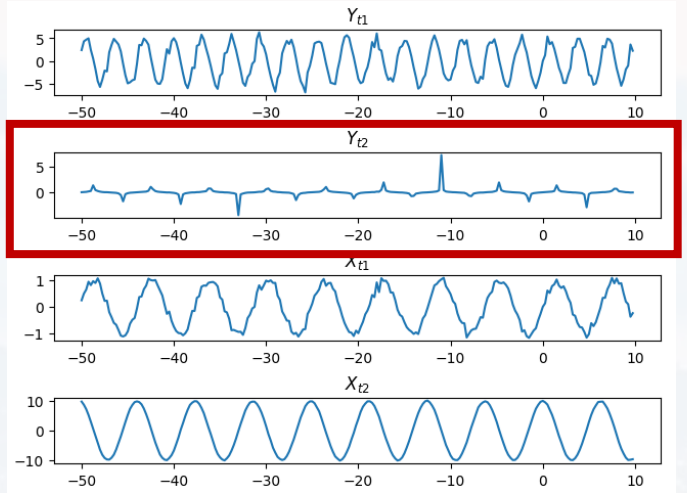


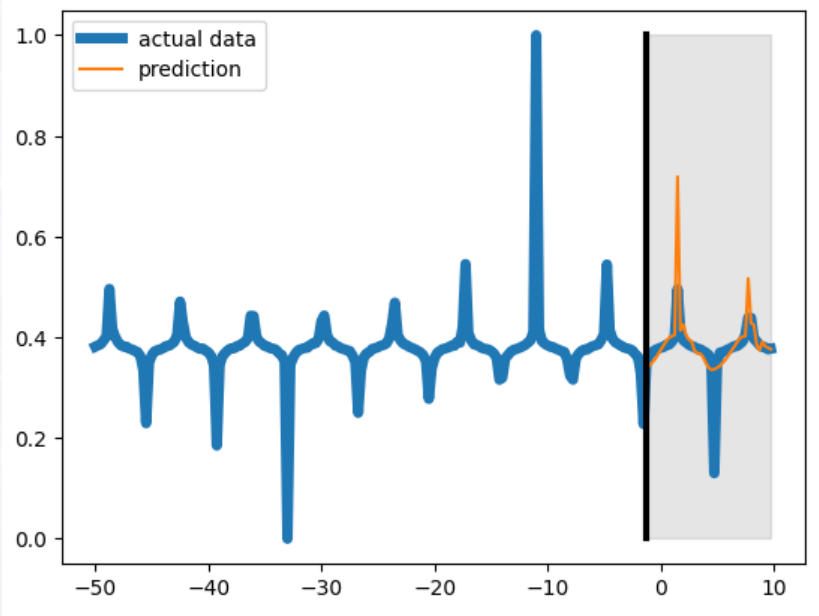


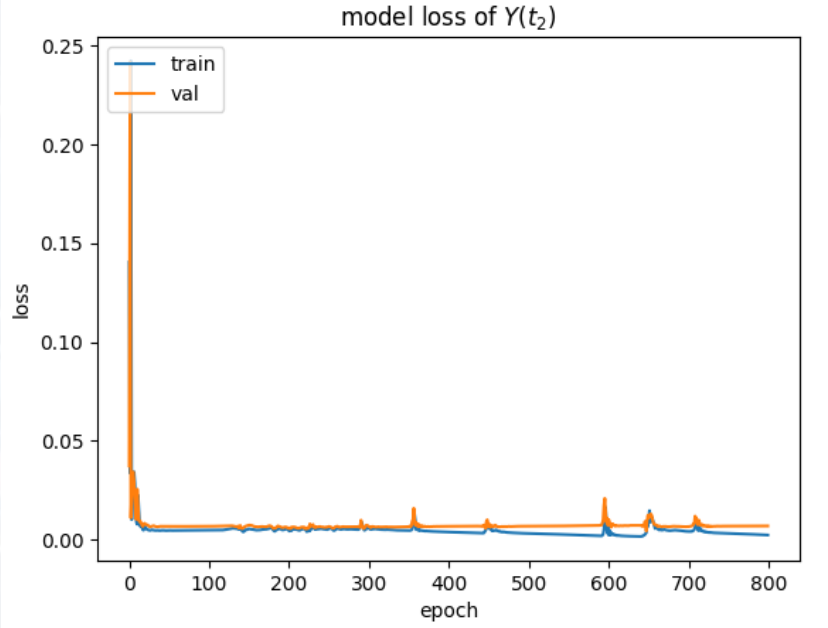
LSTMs:

Syntax and some examples

Explore `LSTMII.ipynb` for a multivariate, multi feature time series:

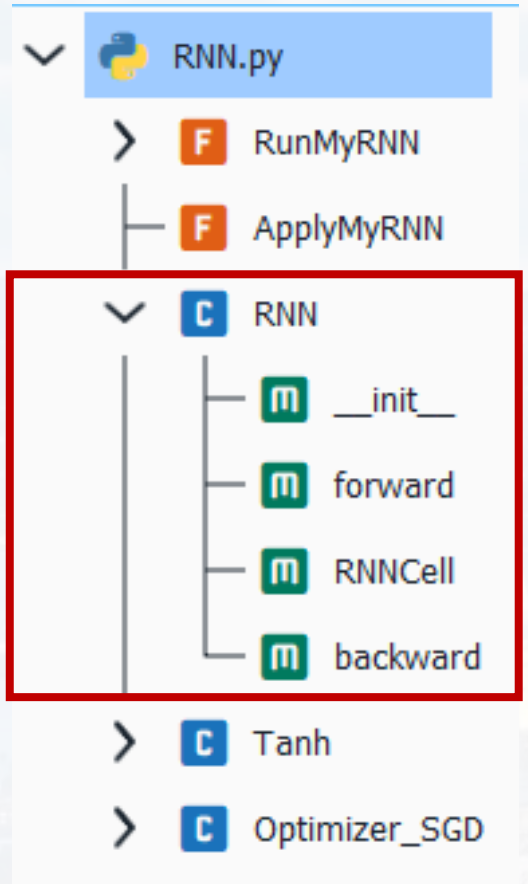


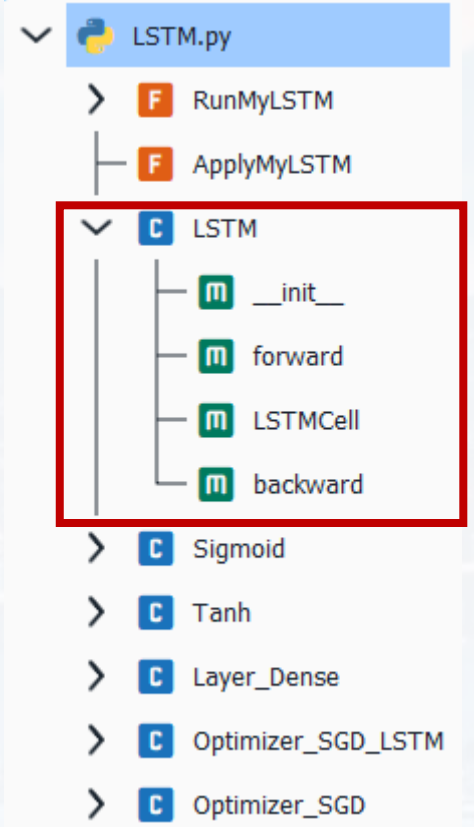




RNNs & LSTMs:

RNN & LSTM spelled out: explore **`LSTM.py`** and **`RNN.py`**





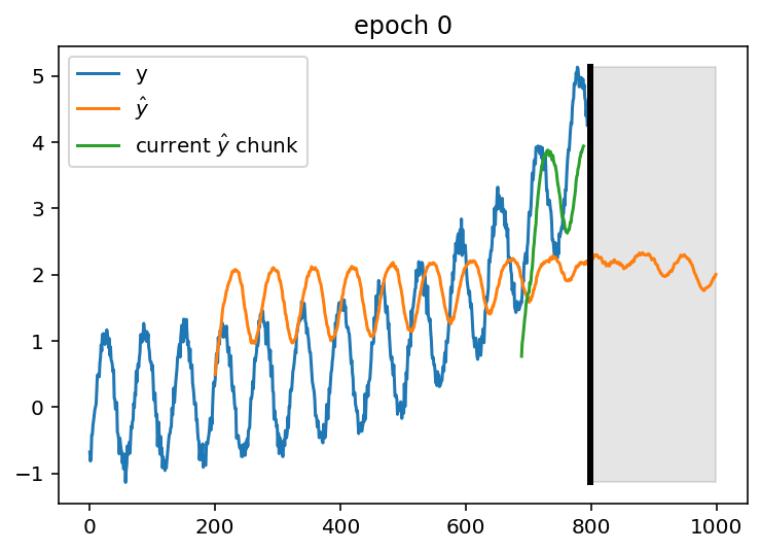
run **`testCell.py`**

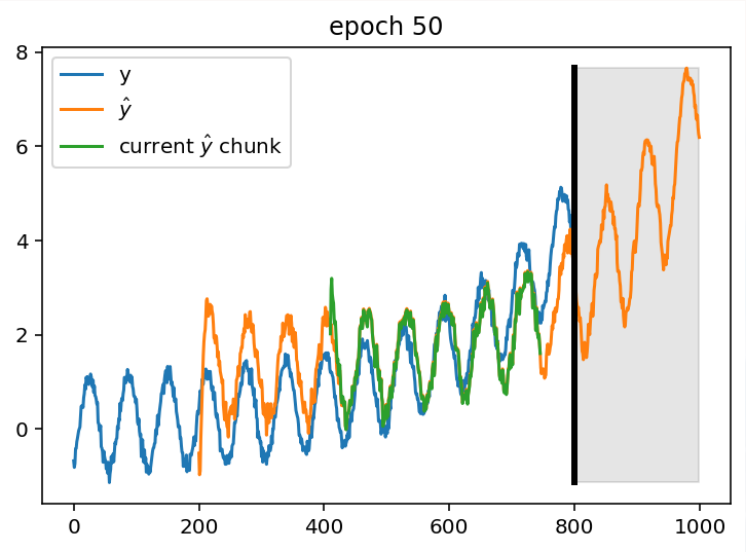
```
LSTM is running...
current MSSE = 0.888
current MSSE = 1.127
current MSSE = 0.619
current MSSE = 0.474
current MSSE = 0.389
current MSSE = 0.364
current MSSE = 0.440
current MSSE = 0.303
current MSSE = 0.366
current MSSE = 0.344
current MSSE = 0.295
current MSSE = 0.269
current MSSE = 0.238
current MSSE = 0.299
current MSSE = 0.191
current MSSE = 0.196
current MSSE = 0.264
current MSSE = 0.203
current MSSE = 0.182
current MSSE = 0.214
Done! MSSE = 0.140
```

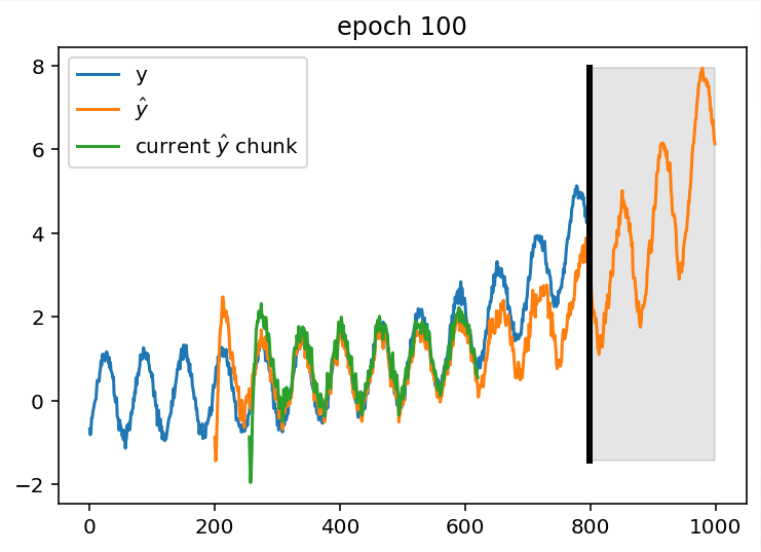
RNNs & LSTMs:

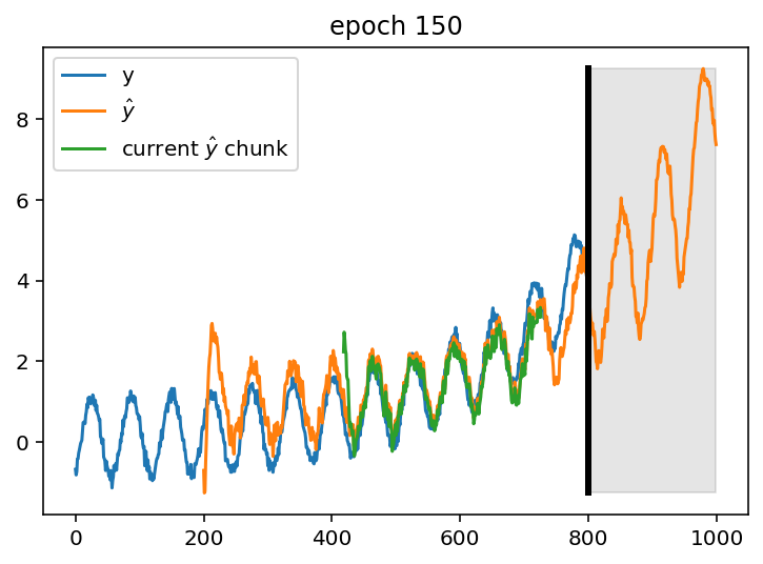
Syntax and some examples

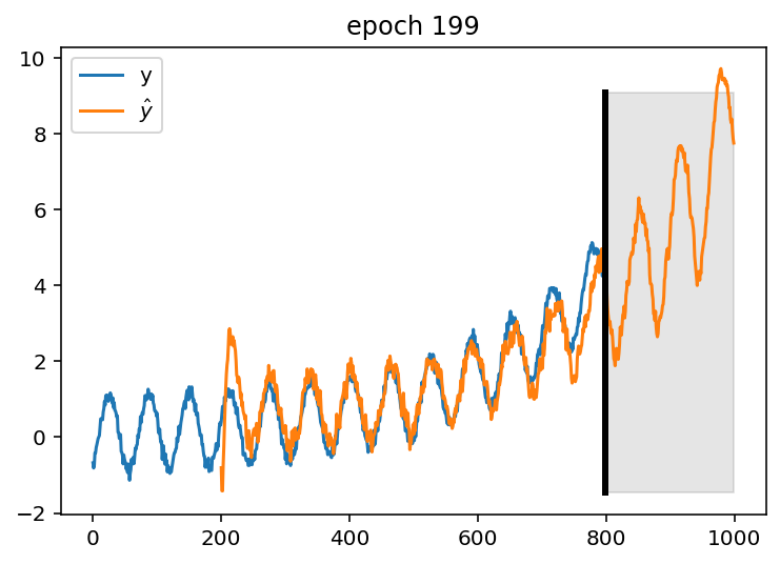
LSTM.py







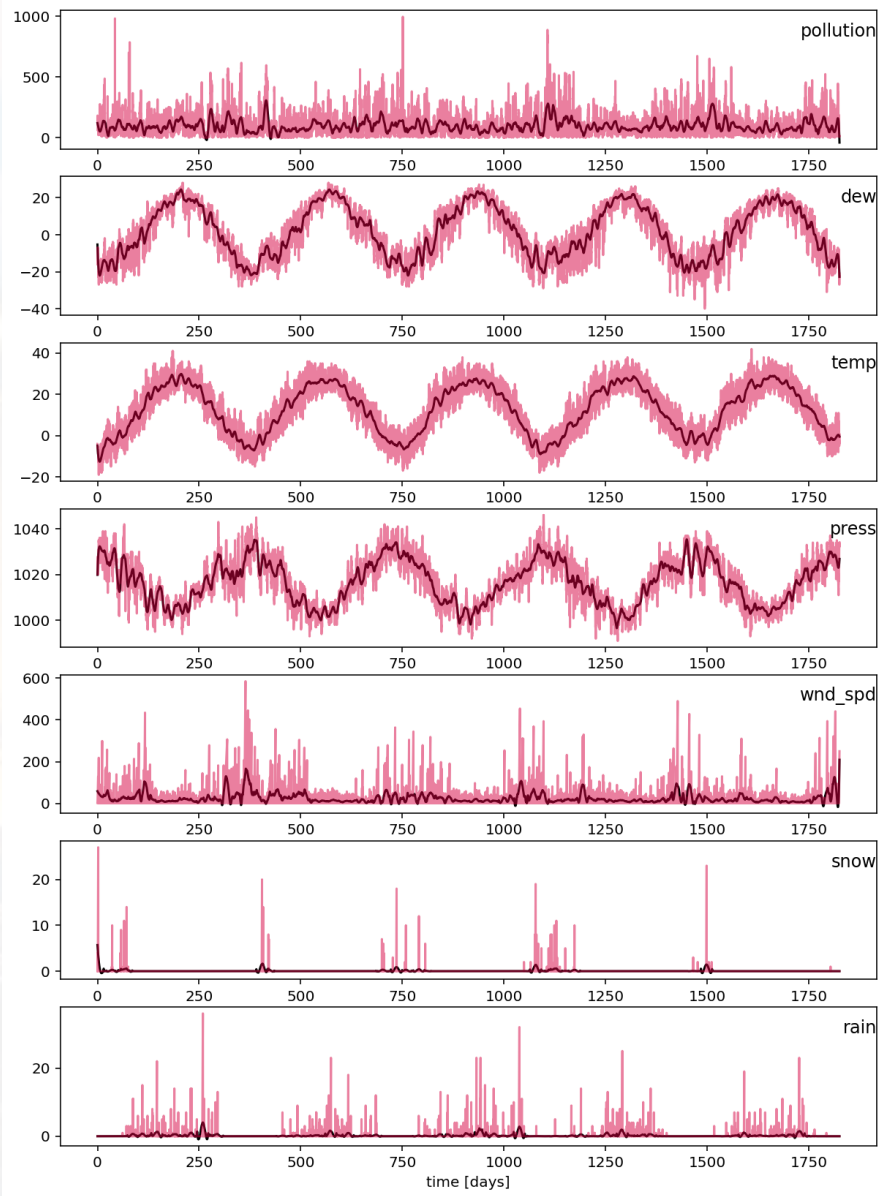


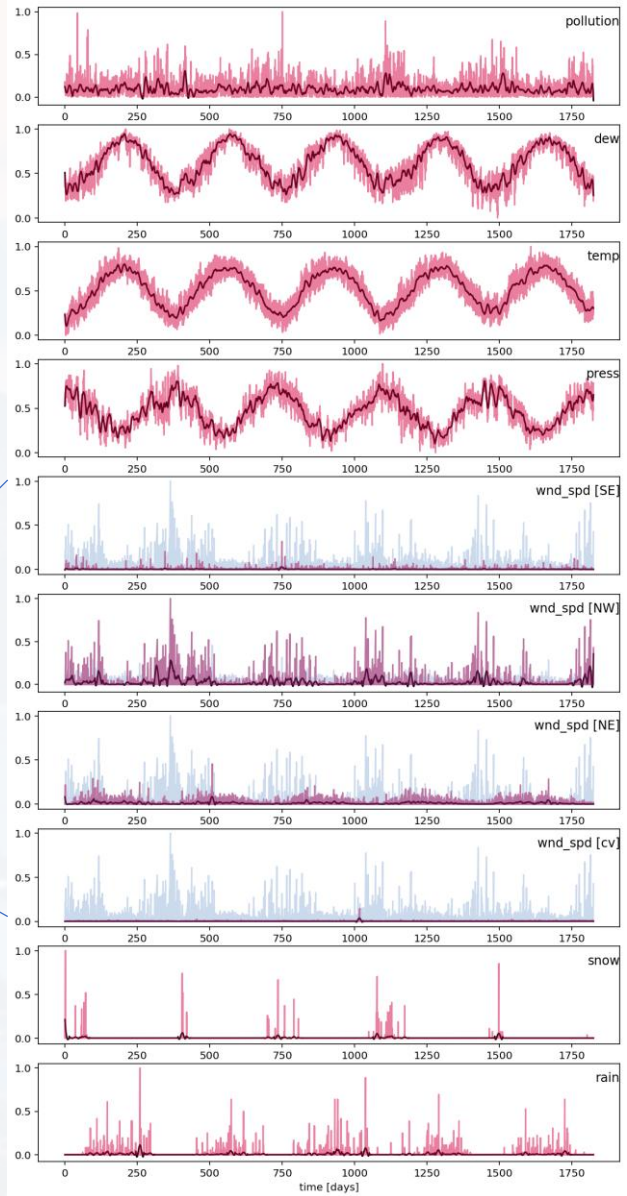


RNNs & LSTMs: Syntax and some examples

`LSTM_keras.py` multi variate example taken from here: forecasting pollution from weather conditions

actual data
monthly moving average

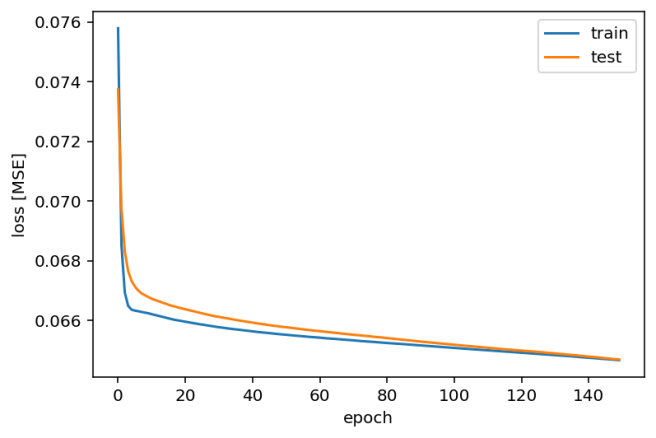


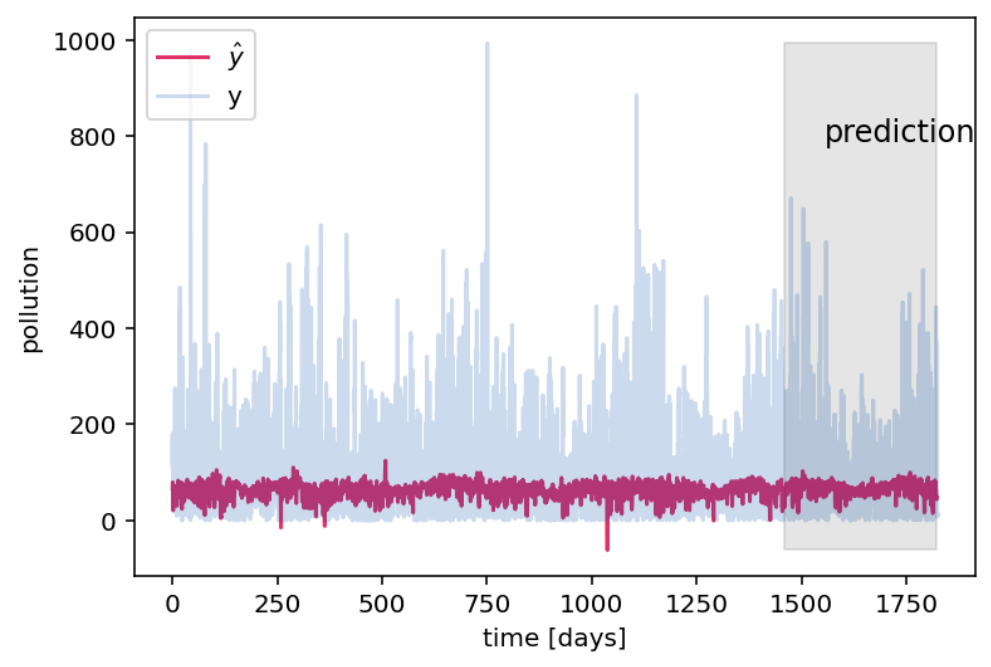


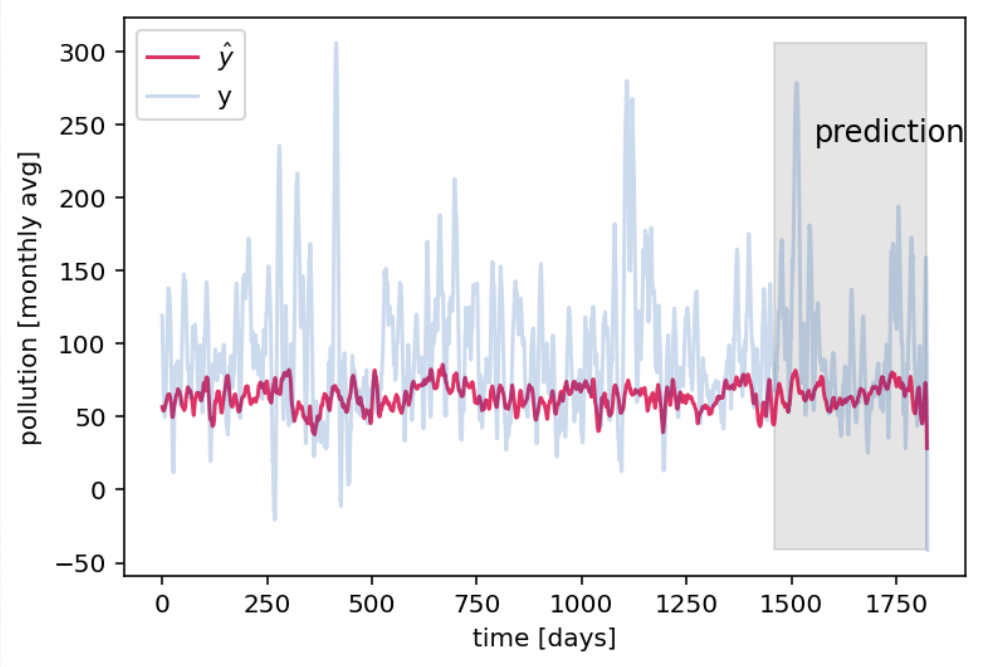
RNNs & LSTMs: Syntax and some examples

`LSTM_keras.py` multi variate example taken from here: forecasting pollution from weather conditions

150 neurons



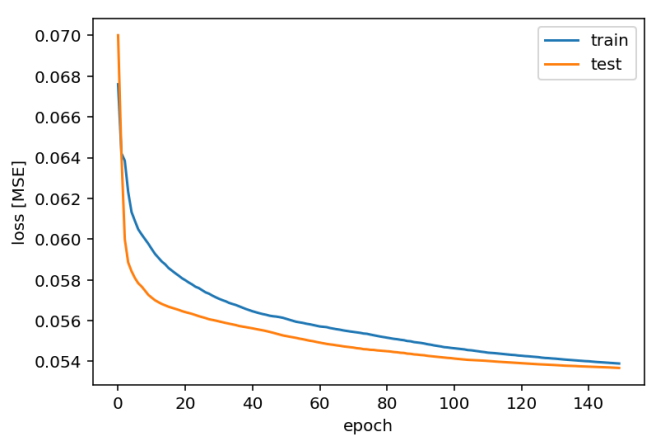


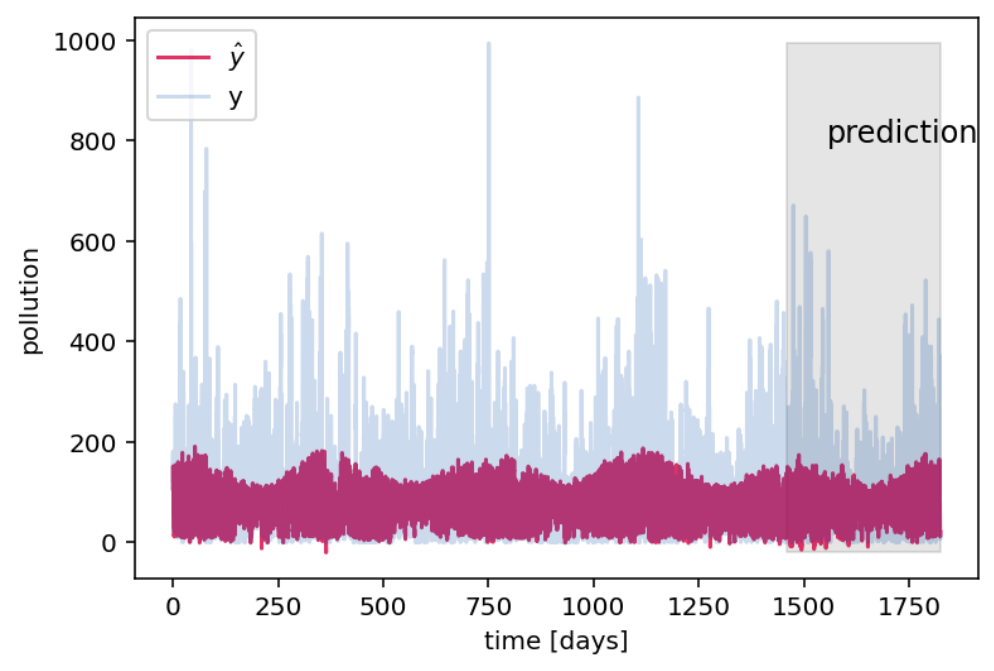


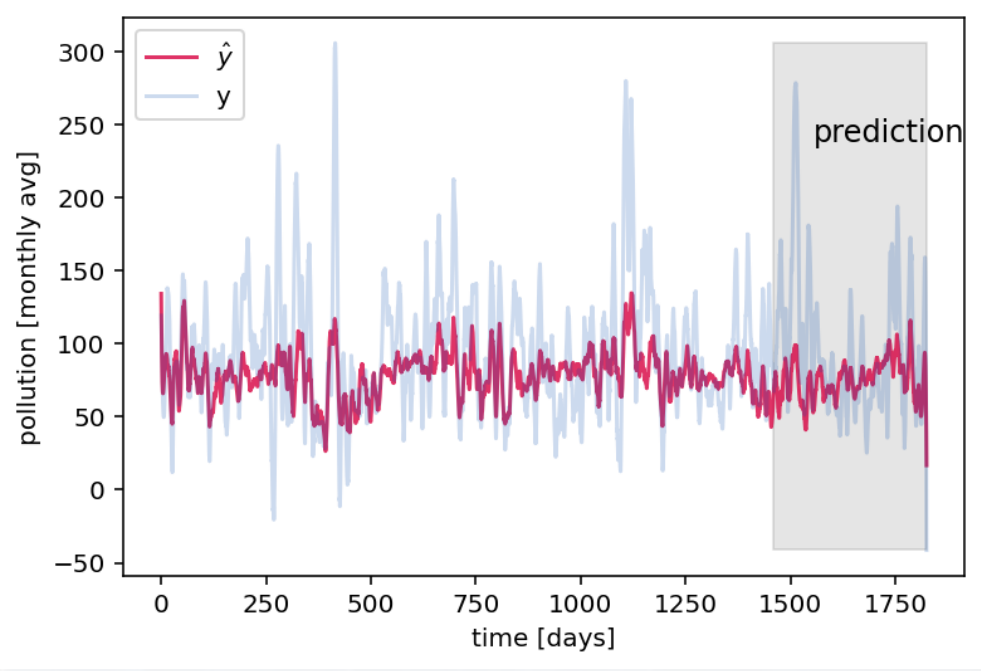
RNNs & LSTMs: Syntax and some examples

`LSTM_keras.py` multi variate example taken from here: forecasting pollution from weather conditions

500 neurons



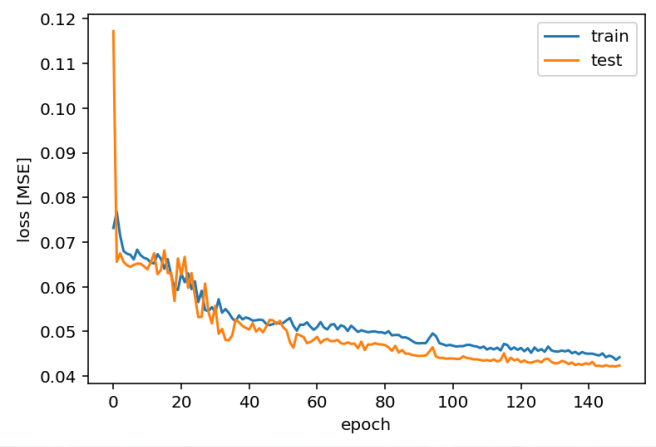


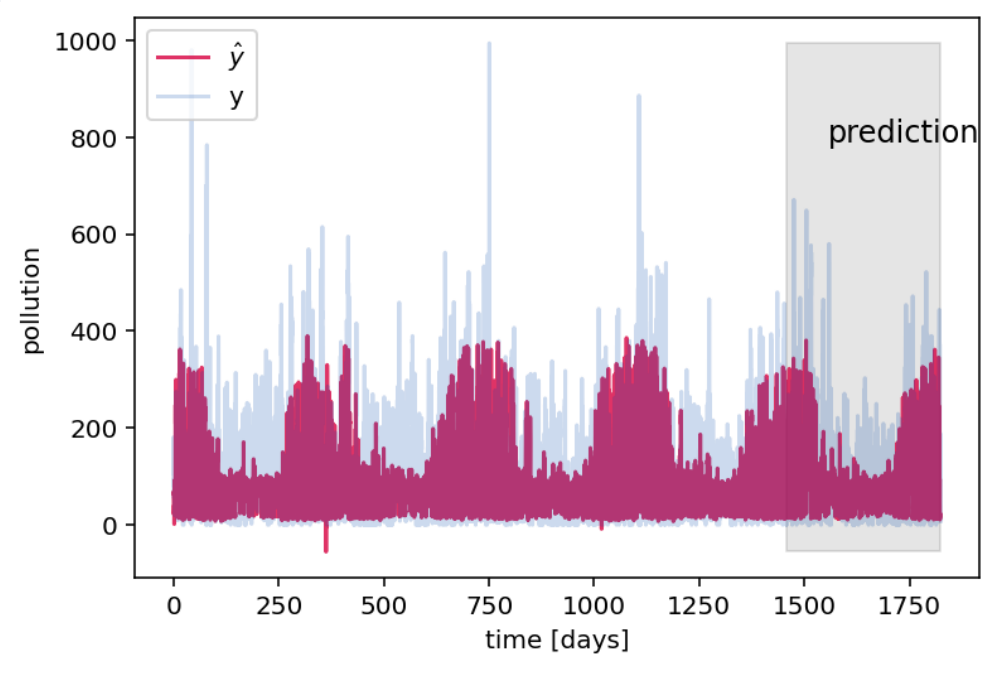


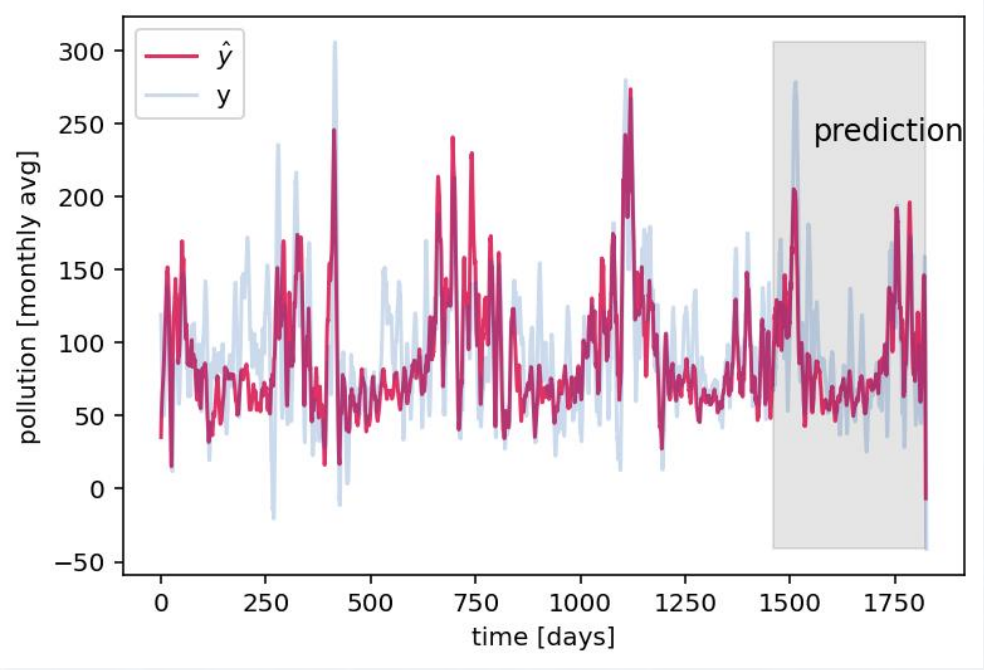
RNNs & LSTMs: Syntax and some examples

`LSTM_keras.py` multi variate example taken from here: forecasting pollution from weather conditions

800 neurons







Thank you very much for your attention!

https://www.analyticsvidhya.com